

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



LE HOANG MINH

**AGENTIC AI TEACHING SLIDES DEVELOPMENT
PLATFORM EDLORA**

BACHELOR'S THESIS

Major: Computer Science

Supervisor: Dr. Nguyen Van Son

Co-Supervisor: Associate Professor Vo Dinh Hieu, PhD

HANOI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



LE HOANG MINH

**AGENTIC AI TEACHING SLIDES DEVELOPMENT PLATFORM
EDLORA**

BACHELOR'S THESIS

Major: Computer Science

Supervisor: Dr. Nguyen Van Son

Co-Supervisor: Associate Professor Vo Dinh Hieu, PhD

HANOI - 2026

Acknowledgements

Assurance

I declare that I am the sole author of this thesis and that it has not been previously submitted for any other academic or professional qualification. I confirm that the work submitted results from my efforts, supplemented by the valuable guidance and support of Dr. Nguyen Van Son and Associate Professor Vo Dinh Hieu, PhD, for the main purpose of graduation. All information, data, and results included in this thesis are original and have not been plagiarized from any other sources. All references used in this thesis are duly cited and listed in the reference section. I take full responsibility for the content and integrity of this thesis, under the regulations of the VNU University of Engineering and Technology.

Hanoi, / / 2026

Student

Le Hoang Minh

Abstract

Cognitive science demonstrates that storytelling and gamification are among the most effective mechanisms for long-term knowledge retention, yet most educators lack the time to integrate these pedagogies into their lessons. Existing tools force teachers to navigate fragmented software ecosystems—using separate applications for brainstorming, content generation, and visual design—while no current platform natively optimizes its output for pedagogical quality. This thesis presents Edlora, a Lesson Development Environment (LDE) that enables educators to generate curriculum-aligned, narrative-driven teaching presentations through natural language interaction. The proposed system combines a Retrieval-Augmented Generation (RAG) pipeline, which grounds all output in mandated textbook content via semantic vector search, with a multi-agent AI workflow orchestrated by LangGraph, where specialized agents collaboratively synthesize retrieved curriculum data into structured, gamified lesson components. A block-based rendering engine decouples content generation from spatial formatting, allowing the AI to produce complex slide layouts without pixel-perfect design. The system is evaluated across three dimensions: automated software testing confirms the reliability of the asynchronous streaming architecture; information retrieval metrics and an LLM-as-Judge framework validate the pedagogical accuracy of generated outputs; and qualitative evaluations with active secondary school educators confirm that the platform reduces lesson preparation effort while elevating instructional quality. The contributions demonstrate a feasible and extensible approach to AI-assisted lesson creation, providing a foundation for future expansion into synchronized lesson plan and assessment generation, multi-vendor model support, and student-level personalization.

Keywords: Lesson Development Environment, Agentic AI, Retrieval-Augmented Generation, Educational Technology.

Tóm tắt

Khoa học nhận thức chứng minh rằng kể chuyện và trò chơi hóa là những phương pháp hiệu quả nhất cho việc ghi nhớ kiến thức dài hạn, nhưng phần lớn giáo viên không có đủ thời gian để tích hợp các phương pháp sư phạm này vào bài giảng. Các công cụ hiện tại buộc giáo viên phải sử dụng một hệ sinh thái phần mềm rời rạc—dùng các ứng dụng riêng biệt cho việc lên ý tưởng, tạo nội dung và thiết kế trực quan—trong khi không có nền tảng nào tối ưu hóa đầu ra cho chất lượng sư phạm. Khóa luận này trình bày Edlora, một Môi trường Phát triển Bài giảng (Lesson Development Environment – LDE) cho phép giáo viên tạo bài trình chiếu giảng dạy bám sát chương trình, được dẫn dắt bởi câu chuyện thông qua tương tác ngôn ngữ tự nhiên. Hệ thống kết hợp một đường ống Sinh tăng cường bằng Truy xuất (RAG) để neo mọi đầu ra vào nội dung sách giáo khoa bắt buộc thông qua tìm kiếm vector ngữ nghĩa, cùng với một quy trình AI đa tác tử được điều phối bởi LangGraph, nơi các tác tử chuyên biệt cộng tác tổng hợp dữ liệu chương trình giảng dạy thành các thành phần bài học có cấu trúc và tích hợp trò chơi hóa. Một công cụ kết xuất dựa trên khối tách biệt việc tạo nội dung khỏi định dạng không gian, cho phép AI tạo bố cục slide phức tạp mà không cần thiết kế chính xác từng pixel. Hệ thống được đánh giá trên ba chiều: kiểm thử phần mềm tự động xác nhận độ tin cậy của kiến trúc truyền phát bất đồng bộ; các chỉ số truy xuất thông tin và khung đánh giá LLM-làm-Giám-khảo xác nhận độ chính xác sư phạm của đầu ra; và đánh giá định tính với giáo viên trung học cơ sở đang giảng dạy xác nhận nền tảng giúp giảm công sức soạn bài đồng thời nâng cao chất lượng giảng dạy. Các đóng góp chứng minh một phương pháp khả thi và có khả năng mở rộng cho việc tạo bài giảng có hỗ trợ AI, tạo nền tảng cho việc phát triển trong tương lai về sinh kế hoạch bài dạy và bài kiểm tra đồng bộ, hỗ trợ đa nhà cung cấp mô hình, và cá nhân hóa ở cấp độ học sinh.

Từ khóa: Môi trường Phát triển Bài giảng, Trí tuệ Nhân tạo Tác tử, Sinh tăng cường bằng Truy xuất, Công nghệ Giáo dục.

Contents

Acknowledgements

Assurance i

Abstract ii

Tóm tắt iii

Contents iv

List of Figures vii

List of Tables ix

List of Abbreviations xi

Chapter 1 Introduction 1

1.1 Context 1

1.2 Problem 3

1.3 Existing Solutions and Limitations 6

1.4 Objectives, Scope, and Strategic Vision 7

Chapter 2 Background and Related Work 9

2.1 Teaching Pedagogies 9

2.1.1 Storytelling 10

2.1.2 Gamification 12

2.2 AI Technologies 14

2.2.1 Retrieval-Augmented Generation (RAG) 14

2.2.2 Agentic Artificial Intelligence (Agentic AI) 15

2.2.3 Large Language Models (LLMs) and AI Infrastructure 16

2.3 Frontend Technologies 17

2.3.1	Rendering Engine and Framework	17
2.3.2	State Management and Validation	17
2.3.3	Editing Interface and UI Design	18
2.3.4	Frontend Testing and Deployment	18
2.4	Backend and Database Technologies	19
2.4.1	Core API Architecture	19
2.4.2	Persistent Database Layer	19
2.4.3	Authentication and Object Storage	20
2.4.4	Backend Testing and Deployment	20
2.5	Continuous Integration and Deployment	20
2.5.1	Code Quality and Testing Workflows	21
2.5.2	Release Management and Git Hooks	21
Chapter 3	Teaching Presentation Development Platform Edlora	22
3.1	Requirements Analysis	22
3.1.1	Use Cases	22
3.1.2	AI Specifications	28
3.2	System Design	35
3.2.1	Overview	35
3.2.2	Multi-Agent Architecture	36
3.2.3	RAG Architecture	50
3.2.4	Backend and Database Architecture	52
3.3	Implementation	57
3.3.1	Use Case: Authentication	57
3.3.2	Use Case: Onboarding	58
3.3.3	Use Case: Lesson Management	59
3.3.4	Use Case: Lesson Creation	60
3.3.5	Use Case: Lesson Presentation	61
Chapter 4	Results and Evaluation	63
4.1	Software Testing Results	63
4.1.1	Procedures and Metrics	64
4.1.2	Results	65
4.2	AI Evaluation Results	67
4.2.1	Procedures and Metrics	67

4.2.2 Results	70
Chapter 5 Conclusion and Future Work	74
5.1 Conclusion	74
5.2 Limitations of the Current Research	75
5.3 Future Work	76
5.3.1 Short-Term Roadmap	76
5.3.2 Long-Term Vision	77
References	79

List of Figures

1.1	Survey results detailing the volume of new teaching slides created by educators in a standard work week.	3
1.2	Primary obstacles preventing educators from authoring their own instructional presentations.	3
1.3	The primary sources from which educators acquire their teaching presentations.	4
1.4	The most significant drawbacks experienced when utilizing pre-made slide decks.	4
1.5	The most time-intensive stages of the manual presentation design process.	5
1.6	The frequency at which educators attempt to integrate storytelling into their slides.	5
1.7	Positioning chart of current educational presentation tools.	6
3.1	Use case diagram illustrating the user journey.	23
3.2	High-level architecture diagram connecting the web client, product API, database, and AI module.	36
3.3	The root graph coordinating the director and developer agents. . . .	38
3.4	The internal workflow of the Lesson Director agent.	39
3.5	The internal workflow of the Lesson Developer agent.	40
3.6	Edlora’s textbook RAG pipeline.	51
3.7	Conceptual relational model mapping users, lessons, scenes, and textbook chunks.	54
3.8	Hosted Clerk sign-in interface for establishing a teacher session. . . .	58
3.9	Main success flow for Clerk session establishment and backend user alignment.	58
3.10	Onboarding wizard for education level and textbook set before accessing the lesson library.	59

3.11 Synchronous onboarding update for education level and textbook set.	59
3.12 Lesson library listing, creation entry point, and per-lesson navigation to the editor.	60
3.13 Lesson list, scaffolded creation, optional delete, and load-for-edit. . .	60
3.14 Authoring canvas with scenes, preview, and assistant chat.	61
3.15 Authoring flow with optional upload, streaming chat with optional textbook grounding, and direct PATCH.	61
3.16 Presentation view for advancing scenes during teaching.	62
3.17 Client-centered presentation with optional lesson refetch before play.	62

List of Tables

3.1	Detailed Use Case Specification: Authenticate (UC-01)	24
3.2	Detailed Use Case Specification: Onboard (UC-02)	25
3.3	Detailed Use Case Specification: Manage lessons (UC-03)	25
3.4	Detailed Use Case Specification: Edit lesson (UC-04)	26
3.5	Detailed Use Case Specification: Present lesson (UC-05)	27
3.6	Node description for the root director entry.	41
3.7	Node description for the root developer entry.	41
3.8	Node description for generation intent detection.	42
3.9	Node description for director routing.	43
3.10	Node description for lesson specification extraction.	44
3.11	Node description for input comprehensiveness assessment.	44
3.12	Node description for plan drafting.	45
3.13	Node description for developer delegation.	46
3.14	Node description for lesson creation.	46
3.15	Node description for the developer report.	47
3.16	Node description for teacher reply composition.	47
3.17	Node description for turn completion.	48
3.18	The core shared state values in the multi-agent architecture.	49
3.19	Catalog of primary HTTP endpoints.	52
3.20	Columns for the synchronized user records.	54
3.21	Columns for top-level lessons edited in the workspace.	55
3.22	Columns for ordered slides or scenes.	55
3.23	Columns for conversational transcripts bound to lessons.	56
3.24	Columns for chunked textbook excerpts and embeddings.	57
4.1	Coverage results ledger for unit and integration suites.	65
4.2	End-to-end requirement validation matrix.	66
4.3	Technologies and tools used for AI evaluation.	68

4.4	Technologies and tools used for RAG evaluation.	69
4.5	Baseline comparison of Vanilla versus Agentic models across evaluation rubrics.	71
4.6	Retrieval evaluation metrics from the benchmark dataset.	72

List of Abbreviations

Abbreviation	Full term
BiLSTM	Bidirectional Long Short Term Memory
BO	Buffer Overflow

Chapter 1

Introduction

This chapter introduces the **motivation** and **structure** of the thesis. First, the context and problem sections explain **how artificial intelligence (AI) changes the fundamental requirements for education**, as well as the heavy workload that prevents teachers from creating the best lessons they can. Next, existing software solutions are analyzed to **show the gap in the current educational technology market**. Finally, the chapter outlines **the vision and scope of Edlora**, an AI platform designed to address these challenges and empower teachers to effortlessly create engaging, meaningful teaching presentations.

1.1 Context

The global economy is changing as AI redefines human labor. Historically, the job market placed **a premium on the mastery of complex functional skills**, such as financial forecasting, legal research, and programming. Today, thanks to advancements in generative AI, machines can perform these tasks quickly and accurately [11]. Human labor is **increasingly less about being good at executing tasks, and more about knowing which tasks are meaningful, and thus, is worth doing**. AI systems cannot do this well, because it is **humans' unique ability to feel that allows them to understand what is meaningful**. For example, a human software engineer can deeply empathize with the fatigue and stress teachers feel when they're developing their lessons late at night. Therefore, the engineer can understand just how important it is to make lesson development software run smoothly and efficiently, so that teachers won't be stressed when it doesn't. Afterwards, the engineer can write and optimize said software, **which can now be done at a speed and precision that once required decades of experience, by leveraging AI**. This example illustrates **AI's capacity to automate a complex functional skill, which in this case is programming, and its inability to truly understand the meaning of that skill, which is**

reducing teachers' stress. Most occupations across the labor market are similarly affected. This is especially true for teachers, who are increasingly less judged by their ability to convey knowledge, and more by the ability to explain the meaning of that knowledge, and thus, inspire students to learn more about it. Since AI will handle many functional tasks across the workforce, **education needs to fundamentally change, shifting its focus from teaching functional skills to helping students understand the meaning of those skills.**

To make this shift, education should use **storytelling**. Storytelling has been practiced by humans since the Paleolithic era (the Old Stone Age). This is because **storytelling is not just a way to convey information, but also to convey the meaning of that information.** Because when we hear stories, we use our imagination to reconstruct the experience talked about by the story, therefore feeling emotions that we would feel if we actually went through the experience. Going back to our example of a human software engineer empathizing with teachers' fatigue and stress when developing a lesson, this means that the engineer can feel those same feelings by listening to the teachers' stories about their nights spent developing lessons, instead of having to spend actual nights developing lessons. It also means that **teachers can teach students about the meaning of the knowledge they're teaching by telling stories about the subject matter.** For example, math teachers can illustrate the meaning of a certain mathematical formula by telling a story about how it was applied to save people from diseases in the 18th century. Therefore, in theory, **to shift education from being skill-based to meaning-based, we just have to create lessons that present knowledge in the form of meaningful stories.**

However, this vision is **blocked by the heavy workload placed on modern teachers.** Educators face numerous administrative tasks, from reporting students' performances to managing their behavior [6]. These duties consume their limited time and mental energy. This directly affects the quality of **the lesson presentation, which is the primary medium for delivering knowledge within the classroom.** As such, it is also the most essential component of a lesson, capable of framing knowledge as meaningful stories, ultimately realizing the vision. The upcoming sections will **detail these pain points, how they're currently being solved, as well as the thesis's proposed solution.**

1.2 Problem

Many teachers are stuck using basic materials because they lack the time to build better ones. To understand this issue, a survey of $n = 48$ local educators was conducted to study how they create presentations. The data shows that teachers have very little free time. Only 29.8% of teachers can make one or two new presentations a week, while 21.3% make zero new slides in a normal week (Figure 1.1). This happens because of logistical limits. About 39.5% of teachers say a lack of time is the main barrier, and 27.9% say they lack the technical skills to use complex design software (Figure 1.2).

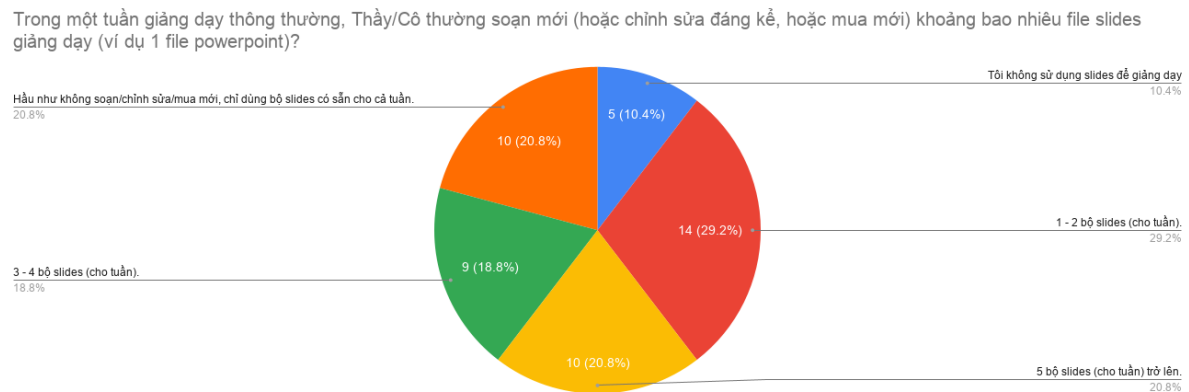


Figure 1.1: Survey results detailing the volume of new teaching slides created by educators in a standard work week.



Figure 1.2: Primary obstacles preventing educators from authoring their own instructional presentations.

Teachers need presentations for their classes, so they must find other ways to get them. They often use secondary sources to quickly build a lesson. The survey shows that only 11.6% of presentations are built from scratch. Instead, 46.5% are minor edits of old personal files, and 27.9% are bought or shared files that need a lot of changes (Figure 1.3). Using these pre-made materials causes problems. Because the materials are generic, 44.4% of teachers report that the slides do not match the cognitive levels of their students. Also, 22.2% state that these mass-produced slides are boring (Figure 1.4).

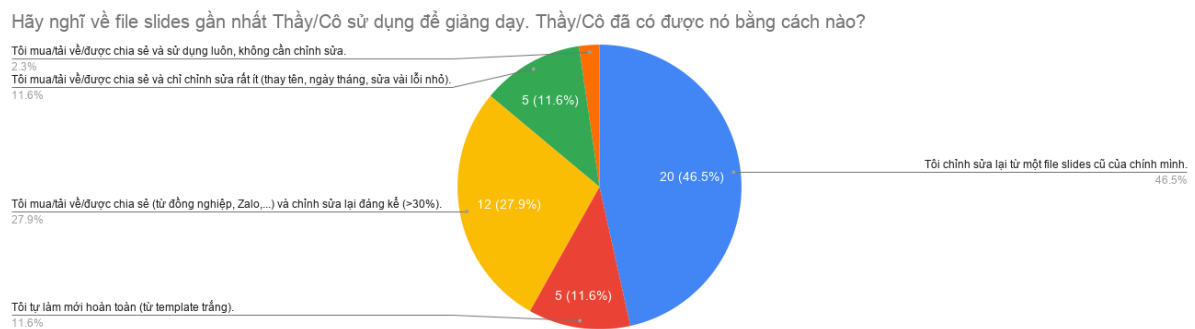


Figure 1.3: The primary sources from which educators acquire their teaching presentations.

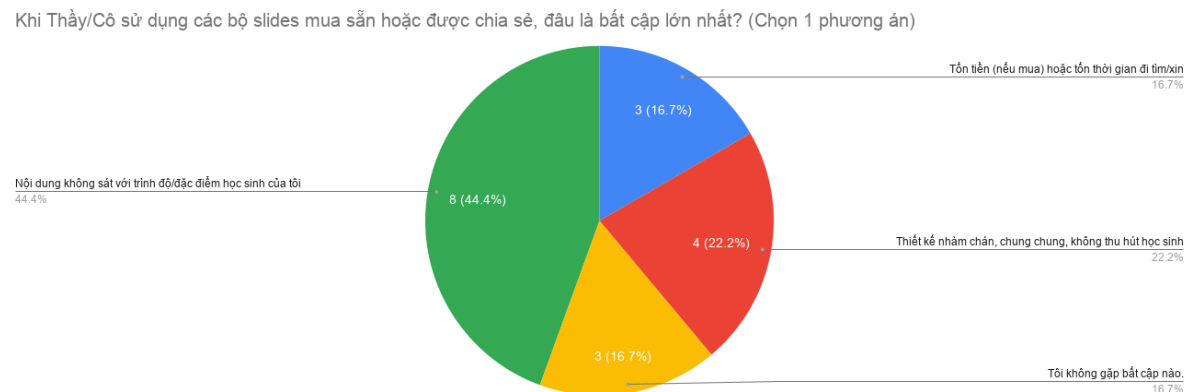


Figure 1.4: The most significant drawbacks experienced when utilizing pre-made slide decks.

To fix this lack of engagement, teachers need the tools to build their own personalized lessons. When looking at the subset of educators ($n = 25$) who actively try to design their own slides, the data shows they are slowed down by modern teaching demands. To keep students interested, 72% of their effort goes into planning the story and flow of the lesson. Another 48% is spent making interactive academic

games, 32% is used to find media, and 32% is spent fixing layouts and colors (Figure 1.5).

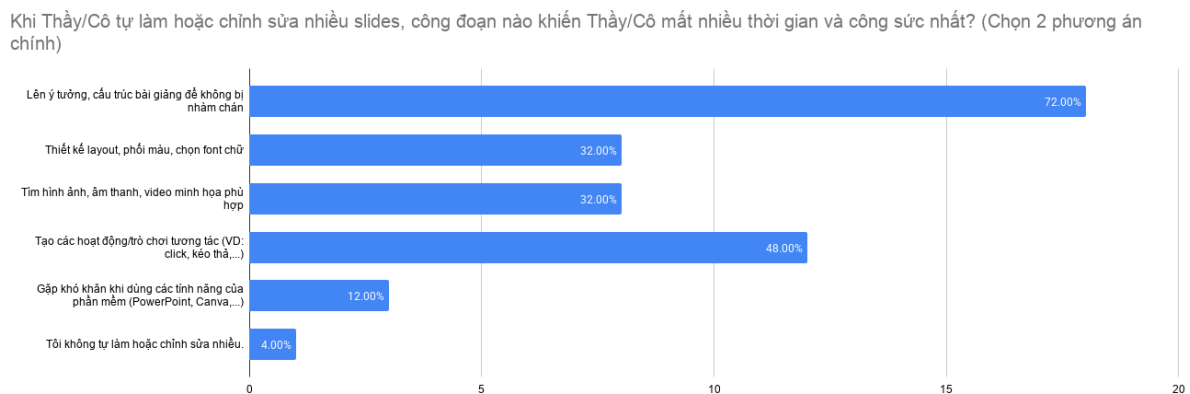


Figure 1.5: The most time-intensive stages of the manual presentation design process.

Teachers know that storytelling and gamification are the best ways to hold student attention. They try to add these features despite the difficulties. The survey of responding teachers ($n = 43$) shows that 53.5% try to use storytelling when they have time, and 27.9% use it frequently. Only one teacher felt these frameworks were not needed (Figure 1.6).

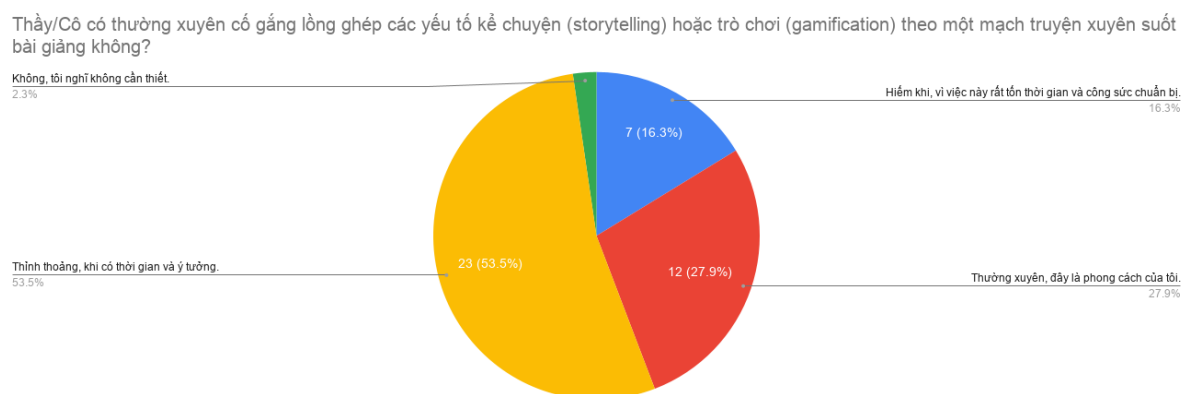


Figure 1.6: The frequency at which educators attempt to integrate storytelling into their slides.

Using current software to solve this time problem only creates new issues. The digital tools available are disconnected. To make a single lesson, a teacher might have to take a photo of a textbook, use an AI chatbot to create a listening test, visit a separate website to make a vocabulary game, and then put all these pieces into a

presentation file manually. This process is fragmented because current educational software is not built around the cognitive science of learning [9]. Since the tools do not guide the teacher, the teacher must use their own technical skills to link the software together. This failure of technology stops teachers from creating the rich lessons they envision.

1.3 Existing Solutions and Limitations

To understand the gap in the market, it is necessary to compare current software tools. This is done by mapping the operational effort the teacher must use against the academic value of the output. Figure 1.7 shows this landscape. The top-right area represents high academic value and low user effort, which is currently empty.

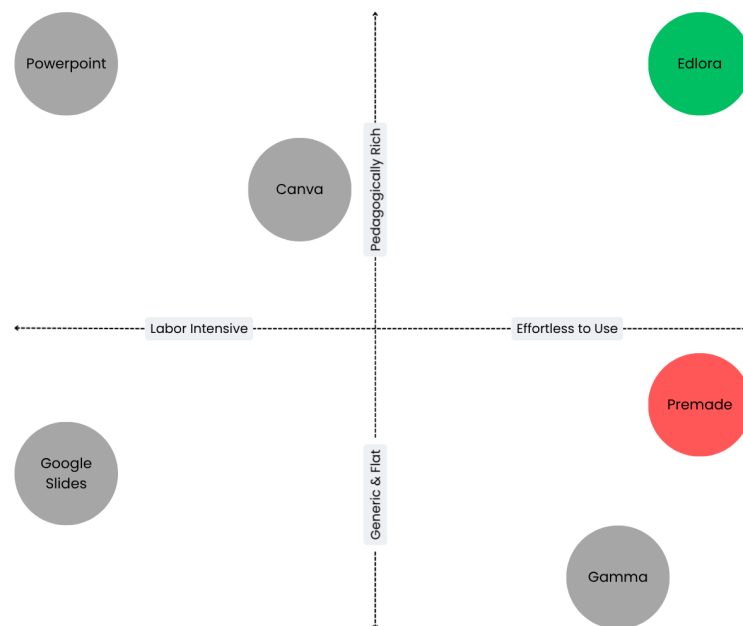


Figure 1.7: Positioning chart of current educational presentation tools.

PowerPoint remains the standard tool for creating deep, interactive lessons. It has complex features that allow teachers to build highly personalized games and slides. However, this manual system requires a lot of time, which most modern educators do not have.

Canva is another popular tool that tries to lower this effort. It provides many pre-designed templates and allows users to embed web games into the slides. This

gives teachers good creative control with less effort than PowerPoint. However, the process is still mostly manual and takes time.

When teachers run out of time, they often buy pre-made lessons. These premium lessons are often built in PowerPoint by experts, so they have high academic value. While they are easy to acquire, they are rigid. This rigidity stops teachers from adjusting the lesson for their specific students.

Google Slides is a basic tool used by many people. It has simple features similar to Canva but lacks a large library of templates. Because it has simple tools but no advanced features, it is not well-suited for building complex, story-driven lessons quickly.

Gamma AI represents new generative AI tools. It focuses on making slides very fast. However, this speed comes at the cost of teaching quality. It uses generic language models that are not trained on teaching methods. Without careful instructions from the user, it creates text that can be incorrect or poorly structured [5]. This lowers the value of the lesson and removes the teacher's control.

This market analysis shows a clear lack of suitable technology. A new system is needed to fill the empty top-right area of the chart. This system must include the story structure and game features that teachers want, while working as fast and easily as modern generative AI.

1.4 Objectives, Scope, and Strategic Vision

To solve this workload crisis, this thesis proposes the design and development of Edlora. Edlora is an Agentic AI Teaching Slides Development Platform. Its main goal is to combine storytelling, teaching science, and modern technology. The core vision of the platform is to turn every lesson into a meaningful journey.

The platform is built on three main rules. First, the "One Lesson, One Evening" rule states that a teacher must be able to make a complete, high-quality presentation in less than one hour. Second, the "One Lesson, One Knowledge Journey" rule ensures every presentation includes story elements and proven teaching frameworks. Third, the "One Creation Method" rule dictates that the system must allow users to create and edit slides using simple tự nhiên ngôn ngữ (Natural Language), removing the need for complex technical skills.

The scope of this thesis covers the end-to-end design and building of this platform. The project aims to show that Retrieval-Augmented Generation (RAG) [7] and Agentic AI can learn teaching rules and generate useful, story-based presentations. The thesis is organized into four main chapters and a conclusion. Chapter 1 introduces the project by discussing the impact of AI on jobs, the heavy workload of teachers, and the limits of current software. Chapter 2 covers the background research, including the science of storytelling and gamification, alongside the modern technologies used to build the platform. Chapter 3 explains the creation of Edlora, detailing how requirements were gathered and how the system architecture was designed and implemented. Chapter 4 presents the results, showing how the software was tested using automated AI checks and real teacher trials. Finally, the conclusion reviews the project's success and lists plans for future updates.

Before building this software system, it is important to understand the science of teaching and the modern computational frameworks that can automate it. The next chapter explores these theories to support the technical choices made in this project.

Chapter 2

Background and Related Work

The development of an agentic AI platform for creating teaching slides requires a multidisciplinary approach, as technological performance depends on the strength of its underlying pedagogical design. Without a solid instructional framework, increased computational efficiency offers limited educational value, merely enabling educators to produce ineffective materials more quickly.

This chapter therefore outlines the theoretical foundations of the Edlora platform and explains the rationale behind its core architecture. It begins by examining relevant insights from cognitive science and established teaching methods that shape the system’s outputs, particularly storytelling and gamification. These well-established approaches show that Edlora’s pedagogical model can be grounded in existing cognitive frameworks, ensuring that the materials it generates are meaningful and effective.

The chapter then analyzes the technological stack required to support the platform, including AI algorithms, frontend systems, and backend infrastructure. By selecting widely adopted, industry-standard technologies with strong developer support, the platform enables rapid development and reliable performance. This integration of pedagogy and technology allows theoretical principles to be translated into an automated and scalable system.

2.1 Teaching Pedagogies

Software engineering in education functions primarily as a delivery mechanism; the effectiveness of a system ultimately depends on the quality of its pedagogical design. To scale high-quality teaching across educators, it is first necessary to understand what makes a lesson effective. This section therefore examines insights from cognitive science to define the instructional frameworks that Edlora is designed to automate.

2.1.1 Storytelling

This subsection examines one of the most fundamental mechanisms of human information transfer, which is storytelling. Invented millennia before the advent of the written word, narrative construction remains the primary method by which humanity has historically conveyed critical information, preserved cultural knowledge, and conceptualized abstract ideas.

Narratives serve a diverse array of evolutionary functions. They are utilized to rapidly communicate existential threats, hypothesize about the physical universe, and articulate complex emotional states. Because storytelling permeates every facet of the human experience, the human brain has evolved to become highly optimized for both the construction and the retention of narrative structures. Education, being the formalization of information transfer, is intrinsically linked to this evolutionary trait. However, understanding why it works requires examining the cognitive encoding process.

There is substantial empirical evidence validating the cognitive superiority of narrative learning. In the foundational 1995 paper, *Knowledge and Memory: The Real Story*, cognitive scientists Roger C. Schank and Robert P. Abelson proposed that human memory is fundamentally organized in the form of interconnected stories [10]. The hypothesis posits that human knowledge is not stored as a flat database of isolated facts, but is constructed from past experiential narratives. Comprehension is mathematically and logically modeled as the process of mapping novel experiences onto existing script indexes within the brain.

Schank and Abelson concluded that the act of telling is synonymous with the act of remembering. When a student encounters a discrete grammatical rule or an isolated vocabulary word, the brain struggles to encode it because it lacks an associative index. However, when that rule is embedded within a narrative, such as a story about navigating a foreign city, the brain utilizes the narrative arc as a cognitive scaffold. Details that fail to integrate into a cohesive narrative structure are rapidly purged from short-term memory to preserve cognitive bandwidth.

More recently, a 2024 study titled *Stories, Statistics, and Memory* conducted by Graeber, Roth, and Zimmermann provided quantifiable metrics comparing narrative versus statistical information retention [2]. The study hypothesized the

existence of a story-statistic gap, suggesting that qualitative narratives yield significantly higher long-term retention rates than quantitative data. Through rigorous experimentation, the researchers presented distinct groups with either statistical data or narrative descriptions regarding an identical subject.

The empirical results were stark and highly relevant to educational design. After a 24-hour period, the retention of statistical data decayed by 73%, whereas the retention of narrative information decayed by only 32%. Furthermore, the absolute accuracy of narrative recall was recorded at 61.6%, more than double the 28.7% accuracy rate observed for statistical recall. This proves a critical algorithmic constraint for Edlora: abstract academic data must be contextualized to survive short-term memory decay. Presenting a student with a sterile list of facts guarantees a massive knowledge attrition rate within a single day.

Finally, the cognitive utility of narratives in chaining sequential information was definitively proven in the classic study *Narrative stories as mediators for serial learning* [1]. The researchers hypothesized that organizing disjointed information into a thematic story creates a robust cognitive structure, mitigating memory interference. Subjects were tasked with memorizing lists of disjointed nouns. The control group utilized standard rote memorization, while the experimental group constructed narrative chains linking the words.

During long-term recall testing, the narrative group achieved a median recall rate of 93%, exponentially outperforming the control group, which achieved a mere 13% recall rate. Remarkably, the lowest-performing subject in the narrative cohort still outperformed the highest-performing subject in the control cohort. The study concluded that narrative chaining effectively creates an artificial semantic network. Retrieving one part of the story automatically triggers the retrieval of the embedded vocabulary.

Synthesizing these empirical findings establishes a formidable foundation for the Edlora architecture. The data dictates a dual insight: storytelling is not merely the fundamental mechanism through which human beings discover purpose, it is also the most cognitively efficient methodology for functional information retention. Consequently, every lesson generated by the Edlora ecosystem is algorithmically mandated to integrate narrative elements. The system is engineered to encapsulate raw educational data within a coherent knowledge journey. By doing so, the

platform actively transcends the dry transfer of mechanical facts. This ensures that the generated artifacts leverage the brain’s natural episodic memory encoding while simultaneously imbuing the academic curriculum with authentic human resonance and existential meaning.

However, while storytelling constructs this profound overarching framework of meaning, maintaining active cognitive engagement throughout a lengthy narrative requires additional interactive mechanisms. Passive listening, even to a deeply meaningful story, eventually induces cognitive fatigue. This leads to the integration of the second foundational pedagogy: gamification.

2.1.2 Gamification

Parallel to the history of storytelling is the human propensity for play. Within the rigorous scope of this thesis, gamification is strictly defined as the systematic application of game-design elements and game principles within non-game contexts. This definition aligns with fundamental game design theory, which posits that the essence of any game is the tension between potential failure and anticipated achievement, governed by a strict set of rules.

By this definition, humanity has leveraged gamified systems throughout its existence, from the calculated risks of early hunting strategies to the complex social dynamics of modern economics. Therefore, it is a logical progression to gamify the learning process. The pedagogical adage asserting that the best way to learn is through practice is fundamentally accurate. However, any form of academic practice can be translated into a gamified structure. This alignment explains the massive trend of gamification within the modern educational technology sector.

This trend is heavily supported by robust scientific literature. In the 2015 study *Gamification: The effect on student motivation and performance at the post-secondary level*, research demonstrated that the integration of points, leaderboards, and badges significantly bolsters student motivation [8]. This is particularly true among digital-native demographics who often exhibit apathy toward traditional didactic pedagogical delivery. The analysis of 19 empirical studies revealed that gamification consistently elevated attendance rates and classroom engagement, with one metric indicating a 12.9% reduction in course failure rates. This success is largely

attributed to the transition from passive information consumption to active participation, which stimulates dopamine release in the brain’s reward pathways.

Furthermore, a comprehensive literature review titled *Does Gamification Work?* validated the positive psychological impacts of gamified learning, while noting crucial caveats [3]. The study concluded that while gamification is highly effective in educational contexts, it is subject to the novelty effect. The psychological impact of extrinsic rewards decays over time once the user normalizes the system. If the underlying educational task is not intrinsically motivating, the gamification layer will eventually fail.

To combat this decay, gamification must evolve beyond superficial mechanics. A 2025 study examining gamification established that game mechanics must be tightly coupled with the academic concepts being taught, a principle known as concept-binding, to yield true cognitive benefits [4]. A system that merely overlays a scoring algorithm onto a standard multiple-choice quiz provides vastly inferior pedagogical value compared to a system where the core game mechanic represents the actual conceptual framework being taught.

For Edlora, which focuses heavily on English language acquisition, the narrative itself acts as the core academic concept. The system utilizes interactive games strictly as a vehicle to tell and advance this narrative. By requiring the student to solve language challenges to progress the story, the game mechanics and the academic curriculum become seamless. This deep integration successfully implements concept-binding, ensuring the learning experience fosters intrinsic motivation and avoids false metaphors.

However, empirical evidence also highlights that gamification is not a universal panacea. An over-reliance on highly competitive mechanics can demotivate certain student profiles and induce cognitive overload. Therefore, unlike the mandatory integration of storytelling, Edlora treats gamification as a modular option. The system consistently presents interactive frameworks to the educator, ensuring that the final decision regarding its deployment remains entirely at the discretion of the teacher, thereby preserving the educator’s pedagogical autonomy.

Systemizing these nuanced pedagogical strategies into an automated platform is impossible using traditional hard-coded software. Executing contextual storytelling and adaptive, concept-bound gamification necessitates a robust foundation

of cutting-edge artificial intelligence technologies.

2.2 AI Technologies

The intelligence layer of Edlora is responsible for generating the core pedagogical payload. Because academic integrity demands absolute accuracy, using ungrounded conversational chatbots is an unacceptable risk. To ensure reliable content generation and complex planning, this section details the specific AI technologies used to safely automate the educator’s workflow.

2.2.1 Retrieval-Augmented Generation (RAG)

The primary operational bottleneck observed during initial market research involves an educator who is forced to photograph a school-issued textbook and upload it to a generic chatbot merely to extract an exercise. This workflow places a significant burden on teachers. At the same time, standard generative language models often perform unreliably, producing incorrect or fabricated responses when tasked with generating content based on localized curriculum materials.

Retrieval-Augmented Generation (RAG) resolves this structural flaw by serving as an intercepting architectural framework [7]. When a user issues a prompt, the system translates the query into a multi-dimensional vector and calculates its geometric distance against an external authoritative database, retrieving the most relevant factual text chunks to inject directly into the context window before the language model generates a response. This geometric distance is calculated using Cosine Similarity, as defined in Equation 2.1.

$$\text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (2.1)$$

Within this formula, A_i and B_i represent the individual floating-point components of the query vector and the document vector respectively, while n denotes the total number of dimensions within the latent space. By forcing the language model to synthesize answers based on these mathematically retrieved chunks rather than its internal weights, the framework successfully eliminates ungrounded hallucinations.

By treating textbooks issued by Vietnam’s Ministry of Education as the authoritative ground truth, Edlora utilizes **MinerU** for advanced document parsing that preserves reading orders, **OpenAI Embeddings** for semantic translation, and **PostgreSQL** paired with the **pgvector** extension to store and query this spatial data. Together, this stack guarantees that all generated educational slides are perfectly aligned with the mandated national curriculum without requiring the educator to perform manual context uploads.

2.2.2 Agentic Artificial Intelligence (Agentic AI)

While semantic retrieval guarantees factual grounding, the creation of a cohesive teaching presentation is a complex, non-linear process that standard single-shot conversational models are incapable of managing. An educator might draft an outline, write a slide, realize the narrative pacing is disjointed, revise the outline, and subsequently generate an academic game, requiring a system capable of managing recursive memory and stateful corrections.

Agentic Artificial Intelligence transforms language models from passive text predictors to active computational systems by providing them with dynamic reasoning frameworks and the autonomy to act [?]. By structuring the system as a Directed Acyclic Graph (DAG) where individual tasks are represented as nodes, the AI is granted the capacity to evaluate its current state, execute a specific tool, and use logical reasoning to route itself to the next optimal node, continuously looping until the overarching presentation is assembled.

To execute this multi-step planning effectively, the platform leverages **LangGraph** as the primary orchestration library to manage stateful routing logic, alongside **LangChain** to standardize tool schemas and API interactions. This specific stack was chosen because it provides the highly controllable infrastructure required to safely persist memory across a lengthy generation cycle. Together, these frameworks empower the system to function as an autonomous director, independently handling the non-linear complexities of lesson creation without demanding constant manual intervention from the educator.

2.2.3 Large Language Models (LLMs) and AI Infrastructure

Digitizing the intellectual process of pedagogical reasoning requires a computational engine capable of deep semantic comprehension. The nuances of storytelling and the logical rigidity required for agentic planning cannot be executed by traditional conditional software logic. Large Language Models resolve this intelligence deficit by utilizing the Transformer neural network architecture, granting the model the capacity to synthesize narratives and execute reasoning based on statistical probabilities.

The project relies on a streamlined selection of models to balance intelligence with efficiency. **GPT 5.4** is deployed as the primary creative engine in production, tasked with executing complex instructions and synthesizing lessons for maximum pedagogical efficiency. Conversely, **GPT 5.4 Nano** operates as a highly cost-effective variant assigned strictly to development, testing, and evaluation pipelines. Its role is to serve as the analytical judge within the system, evaluating outputs without incurring production-level costs.

To guarantee the reliability of these AI services, rigorous testing is implemented using the **pytest** ecosystem alongside **LangSmith** for agent tracing and **OpenEvals** for automated pedagogical grading. Because environmental consistency is critical for machine learning workloads, the AI services are isolated within a dedicated **Docker** container. For deployment, this container is hosted on **Render**. Render was chosen because it provides a fully managed cloud platform that effortlessly scales backend microservices, allowing the AI layer to handle concurrent generation requests without manual infrastructure overhead. Ultimately, this comprehensive infrastructure establishes a highly resilient, autonomous cognitive engine. It guarantees that the heavy intellectual labor of pedagogical reasoning is executed flawlessly in the background, entirely unburdening the educator. However, raw computational intelligence holds no practical educational value if the human teacher cannot comfortably direct and interact with it. To transform these invisible, machine-generated narratives into a tangible teaching experience, the platform must seamlessly bridge the gap between artificial intelligence and human perception.

2.3 Frontend Technologies

While the artificial intelligence algorithms operate as the invisible intellect generating the pedagogical payload, the frontend rendering technologies serve as the vital visual bridge, defining exactly how that content is perceived, manipulated, and finalized by the educator. Because Edlora aims to actively replace traditional canvas-driven tools like PowerPoint and Canva, the selected interface technologies must guarantee absolute structural predictability and entirely eliminate catastrophic rendering errors during rapid AI data streaming.

2.3.1 Rendering Engine and Framework

A declarative, component-based rendering paradigm resolves this structural fragility by utilizing a Virtual Document Object Model (DOM). Instead of expensively manipulating the actual browser interface directly, the system maintains a lightweight mathematical representation in memory, calculating and applying only the exact minimum differences required when new data is received, ensuring a smooth user experience.

To achieve this visual stability, the frontend codebase is strictly typed using **TypeScript** and powered by **React**. This library is structurally enveloped within the **Next.js** framework, providing optimized server-side rendering and resilient routing that guarantees immediate application load times for the educator.

2.3.2 State Management and Validation

Managing asynchronous data across such a dynamic application inherently introduces architectural fragility if handled manually. Centralizing client-side interface states and automating server-side data fetching ensures that the user interface remains synchronized with the database. Consequently, the project deploys **Zustand** for minimalistic client state management, **TanStack React Query** for automated server state caching, and secures all user input configurations using **React Hook Form** rigorously validated by **Zod** schemas.

2.3.3 Editing Interface and UI Design

When an artificial intelligence programmatically injects generated content into a visual canvas, it risks destroying the document layout. Modeling the document as a strict mathematical tree of nodes rather than raw text guarantees formatting preservation. Therefore, the educator’s editing experience is constructed using **TipTap**, a headless block-based rich-text editor, which keeps the visual structure locked in place.

To ensure educators seamlessly adopt the new platform, the visual design system must minimize cognitive friction by replicating familiar interaction patterns. The foundational visual aesthetics are constructed using the utility-first **Tailwind CSS** framework to accelerate styling. Qualitative insights from in-depth teacher interviews, conducted as part of the market research process described in Section 1.2, indicate that educators spend most of their working time within the Google Workspace ecosystem. To align with these existing habits, the platform incorporates Material UI (MUI) to deploy robust interactive elements, which implements Material Design, which is the same design system used across Google’s products.

By adopting this familiar visual and interaction framework, the platform reduces the cognitive effort required to learn a new tool, allowing educators to navigate the interface intuitively based on their prior experience.

2.3.4 Frontend Testing and Deployment

To ensure the interface remains structurally sound during rapid development, a comprehensive testing strategy is employed. **vitest** and **Storybook** are utilized for unit and integration testing, isolating individual components to ensure predictable behavior. **Playwright** executes end-to-end (E2E) tests to validate complete user workflows across simulated browsers, while **Chromatic** provides automated visual regression testing to catch unintended styling changes. The frontend application is packaged within its own **Docker** container and deployed natively on **Vercel**. Vercel was chosen because it is purpose-built for Next.js, providing edge network caching and seamless global content delivery.

2.4 Backend and Database Technologies

Processing thousands of simultaneous, long-running artificial intelligence streaming connections inherently bottlenecks server threads, severely degrading application performance when relying on traditional synchronous web frameworks. An educational tool must remain responsive to the human teacher even while generative tasks execute in the background.

2.4.1 Core API Architecture

To fully contextualize this architectural shift, one must first examine the critical limitations of legacy systems. As illustrated in Figure ??, a traditional synchronous WSGI server assigns a dedicated worker thread to each incoming request. When heavily reliant on slow external I/O operations, such as database queries or AI text generation, these worker threads are forced to sit idle and wait. Consequently, the server quickly exhausts its thread pool, catastrophically blocking all subsequent user requests.

An asynchronous server architecture resolves this computational traffic jam. As detailed in Figure ??, modern ASGI frameworks utilize a central, non-blocking event loop. This architectural design allows the server to concurrently manage multiple heavy input and output operations, effectively processing new, lightweight user requests without waiting for the slower language models to finish their discrete conversational tasks.

The core communication bridge for both the central web backend and the isolated AI services is engineered using **FastAPI**, a modern Python web framework served via the asynchronous **Uvicorn** application server. This architecture natively utilizes **Pydantic** to cryptographically validate every incoming data payload before it is permitted to interact with the underlying database or microservices, ensuring absolute data integrity.

2.4.2 Persistent Database Layer

Permanently storing complex relational user data, customized lesson plans, and structural metadata demands a highly robust storage foundation protected against

injection vulnerabilities and schema corruption. Abstracting raw query languages through an Object-Relational Mapper ensures developers interact with the database using type-safe objects. The persistent data layer operates on **PostgreSQL**, seamlessly integrated via **SQLModel** to eliminate duplicate validation code, and safeguarded by **Alembic** to manage iterative database migrations safely. It also natively integrates the **pgvector** extension to house the semantic data required for the RAG pipeline.

2.4.3 Authentication and Object Storage

Building custom cryptographic authentication systems fundamentally introduces immense security liabilities. Offloading the complex user identity lifecycle to a dedicated zero-trust provider neutralizes these operational overheads. Secure session management is delegated to **Clerk**. Furthermore, the storage of unstructured binary data, specifically user uploaded files and multimedia, is handled by **Cloudflare R2** via the **boto3** library, capitalizing on its zero-cost egress traffic model to ensure absolute economic viability.

2.4.4 Backend Testing and Deployment

Testing the backend logic requires simulating complex network states and database interactions. This is achieved using the **pytest** ecosystem, augmented by **httpx** and **respx** for HTTP mocking, alongside **factory-boy** and **Faker** to generate realistic synthetic test data. The backend services operate within their own dedicated **Docker** container and are deployed alongside the AI services on **Render**. The persistent PostgreSQL database is deployed on **Supabase**. Supabase was strategically chosen because it provides a fully managed, scalable PostgreSQL environment with native vector support, radically reducing the operational complexity of database administration.

2.5 Continuous Integration and Deployment

Managing a multi-container architecture requires a robust automation pipeline to ensure code quality and deployment safety. To orchestrate this, the project

utilizes **GitHub Actions** for Continuous Integration and Continuous Deployment (CI/CD). GitHub Actions was chosen because it natively integrates with the source code repository, providing a frictionless environment to define automated workflow triggers.

2.5.1 Code Quality and Testing Workflows

The pipeline is structured around several core workflows to ensure the codebase remains pristine. The Lint workflow enforces code formatting across the repository, utilizing **ESLint** and the **TypeScript compiler** for the frontend, alongside **Ruff** and **Black** for the Python-based backend and AI services. This ensures strict stylistic consistency before code is ever merged. The Tests workflow automatically executes unit tests via **vitest** and **pytest** on every code push, acting as the primary gatekeeper against functional regressions. Visual fidelity is continuously verified by the **Chromatic** workflow, which reviews the frontend UI components for unintended visual changes.

2.5.2 Release Management and Git Hooks

To maintain a clean versioning history, a Release workflow utilizes **standard-versioning** to automatically generate changelogs, bump semantic versions, and publish official GitHub Releases. Before code even reaches the remote repository, local Git hooks managed by **Husky** execute **lint-staged** and **commitlint**, ensuring that all commits adhere strictly to conventional standards and formatting rules via **Prettier**. While certain gaps exist in the automated pipeline, such as omitting complex Playwright E2E tests from the standard push workflow to reduce CI execution time, this comprehensive automation stack guarantees that the codebase remains stable, verifiable, and structurally sound at all times.

Understanding these cognitive pedagogies and computational frameworks establishes the theoretical foundation for the platform. However, translating this theory into a tangible, scalable solution requires a rigorous software engineering lifecycle. Consequently, the following chapter details the precise methodology used to gather requirements, design the system architecture, and implement the Edlora platform.

Chapter 3

Teaching Presentation Development Platform Edlora

This chapter details the engineering and implementation of the Edlora platform. First, the requirements analysis breaks down the system into specific use cases and artificial intelligence (AI) behavioral boundaries. The system architecture is then defined, outlining the single-agent routing logic and database structures. Finally, the implementation phase demonstrates how these designs are executed to deliver the final application.

3.1 Requirements Analysis

This section outlines **the operational behaviors and system boundaries** required to solve the teacher’s lesson authoring challenges. It is divided into two parts: the user-facing journeys modeled as use cases, and the strict business logic governing the artificial intelligence module.

3.1.1 Use Cases

The sprawling complexity of the lesson creation workflow has been distilled into a formalized set of system interactions. In this system, use cases serve as the foundational contract between the end-user’s operational needs and the system’s technical implementation. Edlora defines **five sequential use cases for teachers**: Authenticate, Onboard, Manage lessons, Edit lesson, and Present lesson. These steps are designed to build upon each other in a typical first-time journey.

Use Case Diagram

The system involves two primary actors: the **Teacher**, who creates and runs lessons, and the **Clerk**, which represents the product’s external sign-up and sign-

in experience. The interactions between these actors and the system define the intended order of work.

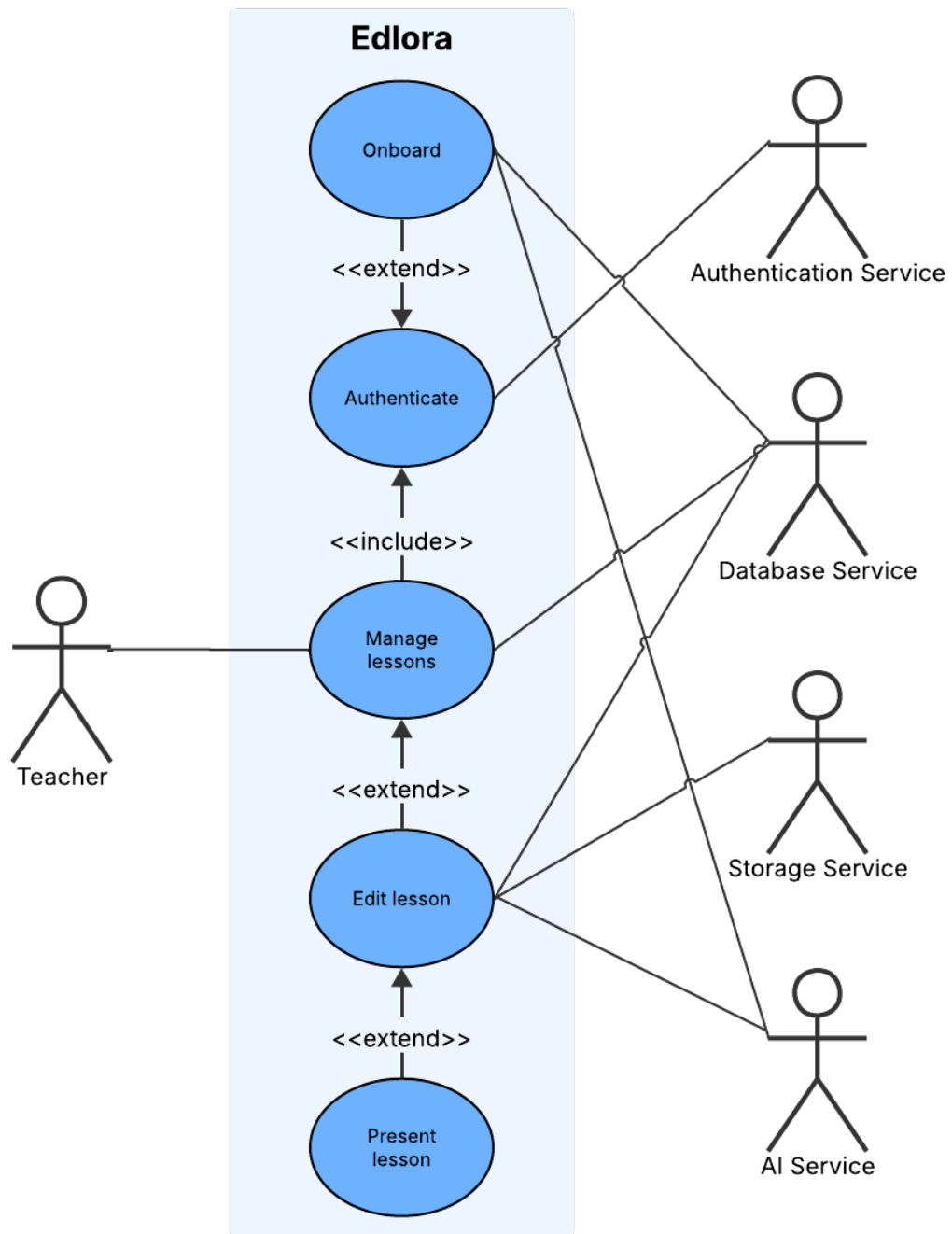


Figure 3.1: Use case diagram illustrating the user journey.

As shown in Figure 3.1, the teacher cannot personalize the product before the system knows their identity. Similarly, the product treats the teacher as ready to use the lesson library only after onboarding is satisfied. Managing lessons unlocks

the deeper work of editing, which in turn prepares the content for meaningful presentation.

Use Case Descriptions

The following tables detail the specific interactions, constraints, and flows for each of the five core use cases.

Table 3.1: Detailed Use Case Specification: Authenticate (UC-01)

Attribute	Description
Use Case ID	UC-01
Name	Authenticate
Actors	Teacher, Clerk (Sign-in experience)
Goal	Let the teacher identify themselves so Edlora can remember their work and enforce strict data isolation.
Prerequisites	None. This is the entry gate.
Trigger	The teacher chooses to register or sign in, or returns with an existing session.
Success Flow	The teacher initiates the sign-up or sign-in process through the hosted identity flow. The system then processes the credentials. Following this, the product successfully recognizes the teacher on later visits. The system allows limited visitor exploration on the home page without a full account, but restricts full lesson work to signed-in users.
Exceptions	If sign-in fails, the teacher remains on public screens or receives an error and is prompted to retry. If the product cannot confirm the account in the background, certain steps are skipped temporarily without completely blocking the user.
Outcomes	The teacher is successfully signed in, and the product associates their subsequent actions with their verified identity.

Table 3.2: Detailed Use Case Specification: Onboard (UC-02)

Attribute	Description
Use Case ID	UC-02
Name	Onboard
Actors	Teacher, Teaching Assistant (AI)
Goal	Capture the target education level and official textbook materials to accurately ground future AI recommendations and chat context.
Prerequisites	UC-01 (Authenticate) is complete.
Trigger	The system detects missing profile fields, or the teacher manually opens onboarding steps.
Success Flow	The teacher selects an education level and a corresponding standard textbook set. The system then plays a short personalization sequence before redirecting the user to the lesson list. Concurrently, the system automatically imports and segments the official textbook content in the background for future search and quoting capabilities.
Exceptions	The system rejects invalid combinations of levels and textbooks with a clear error message. If the background curriculum lookup fails later during chat interactions, the assistant gracefully continues without the specialized textbook context.
Outcomes	The teacher’s profile is populated with level and material data, unlocking full lesson features and textbook-aligned assistant behavior.

Table 3.3: Detailed Use Case Specification: Manage lessons (UC-03)

Attribute	Description
Use Case ID	UC-03
Name	Manage lessons
Actors	Teacher

Table 3.3: Detailed Use Case Specification: Manage lessons (UC-03) (Continued)

Attribute	Description
Goal	View all owned lessons, initialize new drafts, or remove obsolete lessons from the overview list.
Prerequisites	UC-01 (Authenticate) and UC-02 (Onboard) are complete.
Trigger	Opening the lesson list, creating a lesson, deleting from the list, or selecting a lesson to edit.
Success Flow	The teacher views their lessons sorted chronologically. They can create an empty lesson and are subsequently routed to the editor. Alternatively, the teacher selects an existing lesson and confirms its deletion, which affects only their personally owned data.
Exceptions	If the list is empty, the system explicitly encourages the creation of the first lesson. Unauthorized actions trigger an access denial message. Creation conflicts surface as an error explaining the specific clash.
Outcomes	The lesson collection strictly matches the teacher’s intent, with each listed row ready to be opened for detailed editing.

Table 3.4: Detailed Use Case Specification: Edit lesson (UC-04)

Attribute	Description
Use Case ID	UC-04
Name	Edit lesson
Actors	Teacher, Teaching Assistant (AI)
Goal	Transform a partial lesson draft into a story-driven experience containing goals, scenes, media, and interactive chat help.
Prerequisites	UC-03 (Manage lessons) is complete. The user must own the target lesson.
Trigger	Opening a lesson, sending chat messages, attaching files, altering titles, or updating scenes.

Table 3.4: Detailed Use Case Specification: Edit lesson (UC-04) (Continued)

Attribute	Description
Success Flow	The teacher opens a lesson and reviews its current structural state. They interact with the teaching assistant via chat, and the assistant progressively streams replies while simultaneously updating lesson scenes or statuses. The teacher can upload multimedia assets and renames the lesson, manually adjusting scene order or presentation backgrounds. The system persistently saves all modifications to ensure continuity across sessions.
Exceptions	Network or assistant outages result in streamed error boundaries or explicit retry messaging. The system rejects incompatible or oversized file uploads, providing clear explanations. Long processing waits trigger timeouts with guidance to retry.
Outcomes	The lesson successfully reflects the latest chat context, uploaded assets, and structural order intended by the teacher.

Table 3.5: Detailed Use Case Specification: Present lesson (UC-05)

Attribute	Description
Use Case ID	UC-05
Name	Present lesson
Actors	Teacher (as presenter)
Goal	Execute the lesson as a fullscreen slideshow or interactive story for learners, preventing accidental edits during the flow.
Prerequisites	UC-04 (Edit lesson) is complete and the lesson contains meaningful content.
Trigger	The teacher initiates the "Play" or fullscreen preview command.

Table 3.5: Detailed Use Case Specification: Present lesson (UC-05) (Continued)

Attribute	Description
Success Flow	The system transitions the interface into fullscreen presentation mode. The teacher advances the presentation scene by scene using predefined inputs. The system renders narrative text or interactive learning activities based on reusable product templates. Finally, the teacher exits fullscreen to gracefully terminate the presentation session.
Exceptions	If the browser forcibly blocks fullscreen requests, the teacher receives a clear warning and is permitted to retry. Backward navigation is intentionally restricted to prevent the presenter from unintentionally rewinding the live session.
Outcomes	The classroom-facing presentation completes successfully, returning the teacher to the standard editing interface.

In summary, these five use cases map out the foundational user journeys, taking the teacher from initial identity verification through to the live presentation of a lesson. With the user-facing interactions clearly defined, the next section will outline the strict behavioral rules that govern the AI assistant supporting these workflows.

3.1.2 AI Specifications

While the use cases define the overarching user journeys, the core value of Edlora relies on its artificial intelligence module. This module acts as the engine during the lesson authoring process. The AI system is governed by strict business logic that defines acceptable inputs, expected outputs, and non-negotiable structural rules. Drawing inspiration from industry standards for AI model behavior, the following statements define the exact behavioral expectations of the teaching assistant.

Ground responses in verified inputs and maintain truthfulness

By default, the teaching assistant must base its planning and responses strictly on the context actually provided by the system or the teacher. It must never invent

curriculum text, fabricate file uploads, or hallucinate metadata. When instructions conflict, truthfulness about the provided evidence supersedes all other constraints. If official curriculum passages are loaded, the assistant should align its definitions and examples with them. If they are absent, it must not cite fake page numbers. Similarly, it must only acknowledge file attachments that are present in the current chat turn.

Example: Acknowledging file attachments
User: (Sends a message without any image attached) Make the background look like a forest.
<i>Compliant</i> Assistant: I have set the lesson theme to a forest. No background image was attached with this message, so presentation backgrounds will stay as they were. <i>Rationale:</i> The assistant honestly addresses the lack of an attachment while fulfilling the thematic request.
<i>Violation</i> Assistant: Thanks for the forest picture you uploaded! I have added it behind every slide. <i>Rationale:</i> The assistant hallucinates an upload that never occurred, breaking the truthfulness constraint.

Enforce the lesson structure contract strictly

The assistant must ensure that all generated lesson outlines obey strict structural rules to keep the classroom presentation readable and well-paced. A compliant outline must contain between 15 and 20 segments. Furthermore, it must include exactly three specific activity templates: a team confrontation, a vault, and a boss battle. To prevent cognitive overload, the assistant must never place two activities directly next to each other. They must be separated by at least one teaching slide to provide a debriefing moment.

Example: Evaluating an outline draft
User: Put the team confrontation game right after the vault game.
<i>Compliant</i> Assistant: I cannot place two activities back to back because students need a moment to debrief. I will add a short teaching slide between the vault and the team confrontation. <i>Rationale:</i> The assistant prioritizes the hard structural rules over the user's direct request.
<i>Violation</i> Assistant: Sure, I have placed the team confrontation immediately after the vault. <i>Rationale:</i> The assistant blindly follows the prompt and breaks the pacing contract.

Keep teaching slides concise and text-based

To ensure that the presentation content fits the player's screen without scrolling, the assistant must restrict teaching slide text to a maximum of 150 words and 1000 characters. If the source material exceeds this limit, the assistant must split the content across multiple sequential slides. Additionally, the assistant must describe visuals using words rather than relying on embedded image markdown, because the presentation player does not render external images within slide bodies.

Example: Handling a request for a diagram
User: Show a diagram of a plant cell on this slide.
<i>Compliant</i> Assistant: Picture a rectangular plant cell with a rigid outer wall, a large central vacuole, and green chloroplasts scattered inside. Trace each part in the air as you name it. <i>Rationale:</i> The assistant uses descriptive, spoken cues to teach the visual concept.
<i>Violation</i> Assistant: Here is the plant cell diagram: ![plant cell](https://example.com/cell.png). <i>Rationale:</i> The assistant outputs markdown that the system player cannot render.

Require explicit confirmation before building the lesson

The assistant must only transition from drafting an outline to building the playable lesson when the teacher gives a clear, unambiguous command to do so. The assistant must treat ambiguous phrases as a continuation of the planning phase. Building a lesson consumes significant computational resources, so the assistant must clarify ambiguous requests instead of guessing the teacher's intent. Furthermore, the assistant must only accept a build command if the current outline fully satisfies the lesson structure contract.

Example: Confirming the build command
User: This outline looks okay, maybe we could generate something from it soon?
Compliant Assistant: I am glad you like the outline. When you are ready, please tell me explicitly to build the full lesson from this plan now. <i>Rationale:</i> The assistant correctly identifies the ambiguity and prompts the user for a definitive command.
Violation Assistant: Great! Generating the full playable lesson now. <i>Rationale:</i> The assistant wrongly initiates an expensive background process based on vague language.

Ground responses in retrieved curriculum evidence

To further assist teachers, the system utilizes a retrieval-augmented generation mechanism, commonly referred to as RAG. The primary purpose of this module is to **improve lesson-authoring assistance by grounding the AI’s answers in official curriculum or textbook material**. When the teacher asks for help based on an English textbook unit, the RAG module automatically extracts the entire exercise content and seamlessly inserts it into the AI’s active context. However, the assistant must **combine this retrieved context with other lesson signals, rather than replacing them**. It must prioritize the teacher’s active objective and never override explicit constraints already established in the conversation. When official passages are successfully retrieved, the assistant should prefer them for providing definitions, examples, and lesson sequences. Furthermore, it must **provide traceable attribution at a highly useful granularity**, explicitly citing the specific passages it relied upon, and correctly correlating them only to the evidence actually retrieved.

Example: Handling missing retrieval evidence
User: Quote the textbook definition for this grammar point.
<i>Compliant</i> Assistant: I could not retrieve the textbook passage right now. I can give a provisional definition, or retry with a more specific chapter or unit. <i>Rationale:</i> The assistant honestly admits its inability to find the evidence and offers a safe alternative.
<i>Violation</i> Assistant: The textbook says: '...' <i>Rationale:</i> The assistant presents a confident answer with fabricated authority when no evidence was actually retrieved.

Maintain transparency regarding knowledge limits

In a real-world educational environment, curriculum evidence will not always be perfectly available. Therefore, the assistant must **remain useful even when retrieval is missing, partial, or marked by low confidence**, while staying strictly honest about what evidence is actually present. If the system fails to retrieve sufficient evidence, the assistant must **explicitly acknowledge these limits**. It should then either ask a clarifying question, provide bounded guidance with stated uncertainty, or simply state that reliable evidence is unavailable. Under no circumstances should the assistant **silently fall back to confident, unsupported statements**. Even if the entire retrieval service is temporarily unavailable, the assistant must **communicate this as a state limitation**, continuing to help with standard conversation context rather than disguising the outage as a successful, grounded interaction.

Example: Managing partial retrieval
User: Use Unit 4 sequencing and design a quick practice.
Compliant Assistant: I found partial Unit 4 guidance on sequencing. I will align to that excerpt and mark assumptions where the passage is incomplete. <i>Rationale:</i> The assistant uses the available evidence while remaining transparent about its gaps.
Violation Assistant: Unit 4 requires steps A, B, and C exactly. <i>Rationale:</i> The assistant asserts a sequence as authoritative without possessing sufficient retrieved evidence to back the claim.

Align to a strict precedence hierarchy

Because the lesson creation workflow is highly complex, the AI will frequently receive competing instructions from the teacher, the RAG module, and its own system prompt. To resolve these conflicts reliably, the assistant must **evaluate its instructions against a strict precedence hierarchy**. The absolute highest priority is **truthfulness about evidence**, meaning the assistant must never fabricate citations or pretend to have access to missing uploads. The second priority is enforcing the **lesson structure and safety constraints**. The third priority is following the **teacher’s current objective and procedure**. The fourth priority is adhering to the **retrieved official curriculum passages**. Finally, the lowest priority is maintaining a **creative tone and storytelling choices**. This hierarchy ensures that the RAG mechanism strengthens the assistant’s pedagogical alignment, but **never authorizes misleading or hallucinated claims**.

Evaluate and monitor retrieval quality

Finally, to guarantee the system’s ongoing reliability, the RAG module’s operational performance must be **continuously evaluated and monitored**. The system relies on measuring both **answer faithfulness and answer relevance**. Answer faithfulness tracks whether the claims shown to the teacher are genuinely

supported by the retrieved evidence. Answer relevance measures whether the AI’s response actually addresses the teacher’s original intent. In production, the system must **retain enough trace data to debug failures**, securely capturing the user’s intent, the retrieved evidence set, and the final response shown to the teacher. This operational monitoring helps developers detect dangerous structural drift patterns, such as a rising risk of unsupported claims or a frequent, unnecessary reliance on fallback behaviors.

In summary, this section establishes the fundamental behavioral constraints of Edlora’s artificial intelligence module. By enforcing truthfulness, strict structural adherence, explicit user confirmation, and highly transparent retrieval augmentation, the system ensures that AI-generated content remains reliable and pedagogically sound. With both the user workflows and the AI boundaries clearly defined, the next section details the system design and architectural choices required to implement these capabilities.

3.2 System Design

To satisfy the stringent behavioral requirements delineated in the use case analysis, the Edlora system necessitates a highly sophisticated, distributed architecture. The objective of the system design phase is to architect a scalable, fault-tolerant environment capable of simultaneously managing intensive artificial intelligence computations, stateful real-time data streaming, and dynamic client-side rendering. This section provides an exhaustive analysis of the architectural pillars that construct the Edlora Lesson Development Environment: the Deployment Topology, the Artificial Intelligence Architecture, the Database Schema, and the Rendering Model.

3.2.1 Overview

This subsection gives a **whole-system map** of Edlora before the architecture is unpacked in more specialized subsections. The goal is to name the deployable parts, show how requests cross boundaries, and state which component owns persistent data.

Edlora is split into separate deployable applications. The **web client** is built with Next.js [?] and serves as the interface for teachers and students. The **product API** is a FastAPI [?] service that exposes backend resources, enforces access rules, runs domain workflows, and interacts with the database. The **AI module** is an isolated FastAPI service that exclusively hosts the LangGraph lesson workflows.

Figure 3.2 summarizes the runtime wiring. The browser loads the web application and communicates with the product API via JSON over HTTPS. Lesson chat utilizes **server-sent events** (SSE) to stream generated content [?]. The product API validates user identity using Clerk [?] and manages PostgreSQL database sessions. Large binary uploads are routed to Cloudflare R2 via an S3-compatible client, while the database only stores the resulting URLs. To ensure security, the browser never receives the AI service’s API keys. Instead, the backend constructs the lesson payload and calls the AI module server-to-server.

Figure 3.2: High-level architecture diagram connecting the web client, product API, database, and AI module.

Before the lesson graph executes, the backend may retrieve textbook snippets, format them into a unified **textbook context**, and attach them to the AI module’s request. This retrieval-augmented generation (RAG) mechanism [?] keeps the AI grounded in the curriculum. The subsequent sections will document these interconnected platforms, detailing the multi-agent workflows, the RAG pipeline, and the backend data models.

3.2.2 Multi-Agent Architecture

The multi-agent architecture is the engine that transforms a teacher’s request into a structured, playable lesson. It is built around LangGraph, which models an agent workflow as a graph of nodes, edges, and shared state [?]. A node represents a discrete step of work, an edge dictates the execution order, and the state preserves lesson information across the lifecycle. This orchestration is necessary because lesson generation is a multi-step cognitive process: the system must understand the intent, evaluate information readiness, draft a plan, generate scenes, and summarize the result.

Graph Structure

The architecture utilizes one root graph and two specialized subgraphs. The root graph acts as the primary coordinator. It initially routes the teacher’s input to the **Lesson Director**, which comprehends intent, drafts creation plans, and determines generation readiness. If the plan is approved, the root graph delegates execution to the **Lesson Developer**, which transforms the plan into concrete presentation scenes. Finally, control returns to the director to compose a human-readable response.

This separation of concerns ensures that the director focuses on pedagogy and conversational alignment, while the developer focuses strictly on content materialization.

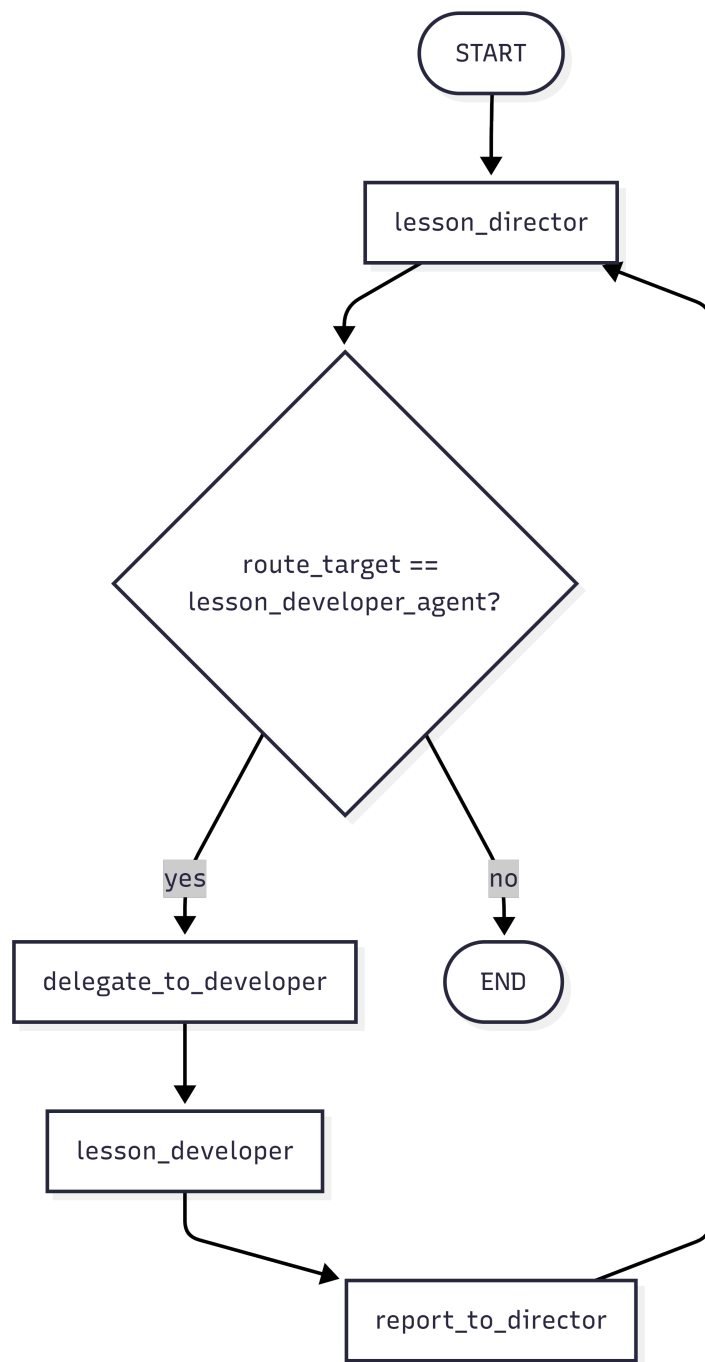


Figure 3.3: The root graph coordinating the director and developer agents.

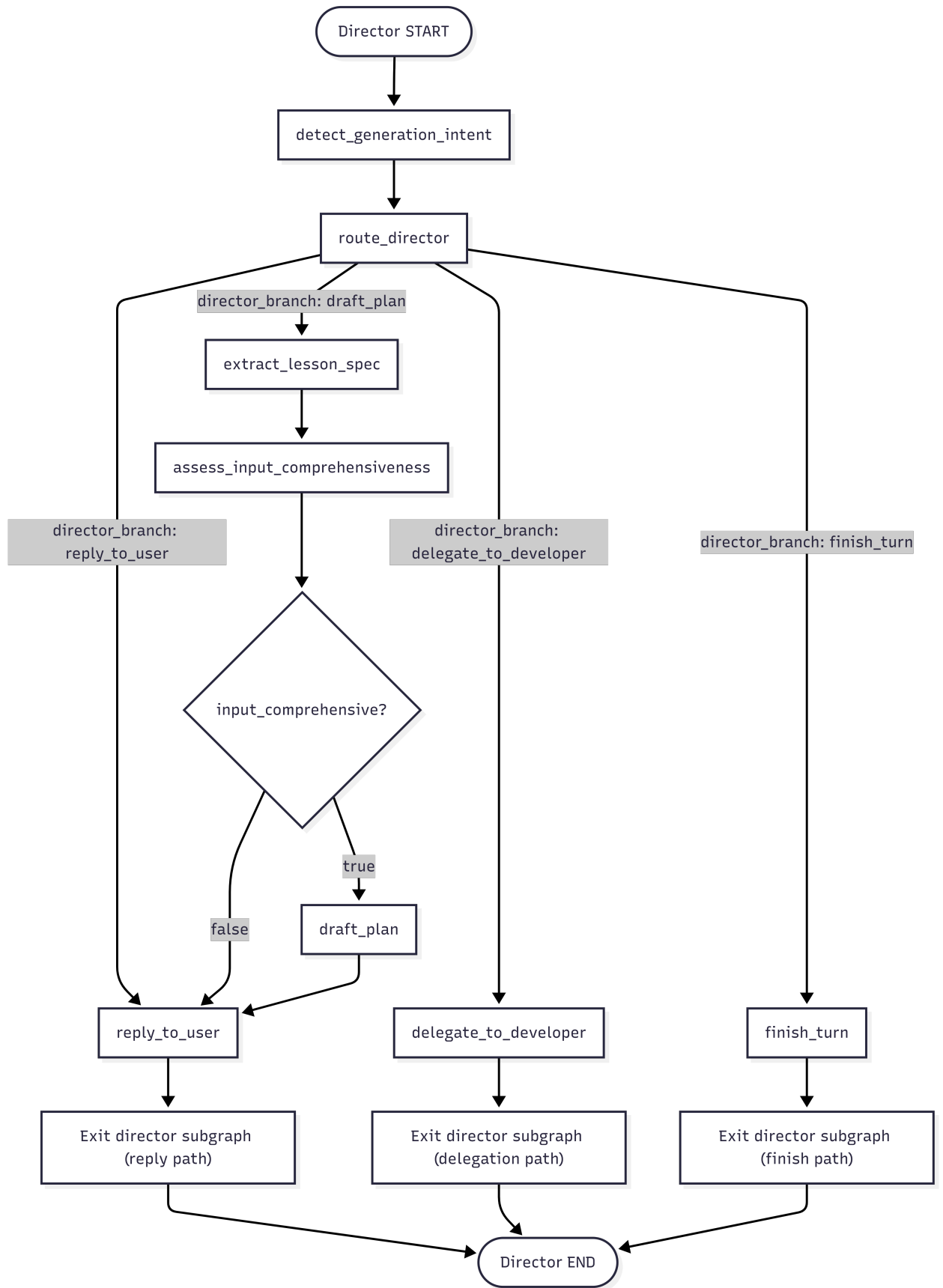


Figure 3.4: The internal workflow of the Lesson Director agent.

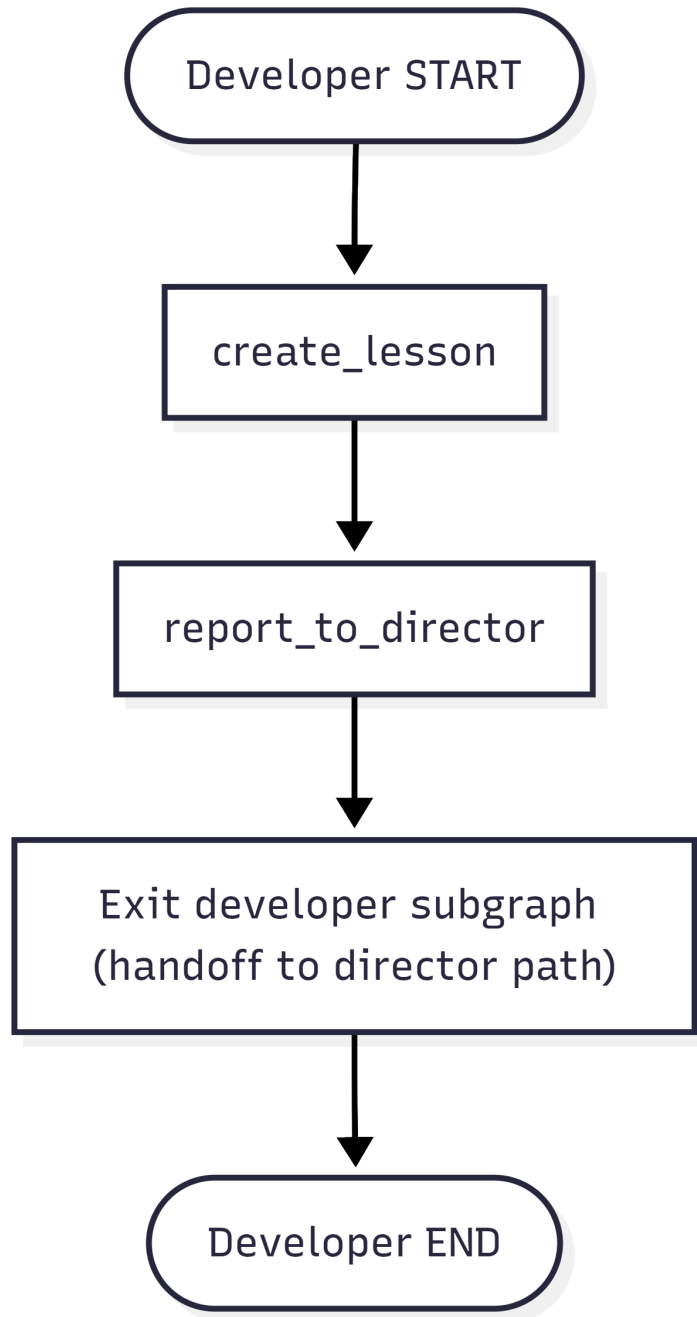


Figure 3.5: The internal workflow of the Lesson Developer agent.

Inside the director subgraph, the system extracts the lesson specification, checks if the context is comprehensive, and drafts a plan. If the teacher merely asks a question, the director bypasses planning and replies directly. The developer subgraph operates sequentially: it receives the plan, generates the scenes, validates the output, and reports back to the director.

Node Responsibilities

The runtime graph contains 12 active nodes. The tables below outline the core responsibilities, execution triggers, and recovery logic for each node in the system.

Table 3.6: Node description for the root director entry.

Field	Description
ID	lesson_director
Agent	Root graph
Role	• Invokes the compiled Lesson Director subgraph. • Acts as the entry point into all director-related behavior.
Execution Trigger	• Runs immediately when the AI graph is called. • Runs again after the Lesson Developer finishes to prepare the final response.
Context & Outcomes	• Forwards the full lesson state, conversation history, and durable memory. • Returns the director subgraph's state update back to the root graph.

Table 3.7: Node description for the root developer entry.

Field	Description
ID	lesson_developer
Agent	Root graph

Table 3.7: Node description for the root developer entry (Continued)

Field	Description
Role	• Invokes the compiled Lesson Developer subgraph to build scenes.
Execution Trigger	• Runs only when the director explicitly routes the turn to the developer.
Context & Outcomes	• Forwards the validated creation plan. • Returns the generated scenes, the developer report, and hands control back to the director.

Table 3.8: Node description for generation intent detection.

Field	Description
ID	<code>detect_generation_intent</code>
Agent	Lesson Director
Role	• Decides whether the latest teacher message explicitly requests generation. • Operates conservatively to prevent accidental generation during casual planning.
Execution Trigger	• Runs as the entry point of the director subgraph.

Table 3.8: Node description for generation intent detection (Continued)

Field	Description
Recovery	• If the structured model call fails, the node retries with a recovery marker. • If all recovery fails, the node defaults to a safe, no-generation intent.

Table 3.9: Node description for director routing.

Field	Description
ID	<code>route_director</code>
Agent	Lesson Director
Role	• Chooses the next director branch based on the detected intent. • Routes to either a teacher reply, lesson planning, or developer delegation.
Execution Trigger	• Runs immediately after generation intent detection.
Recovery	• Runs deterministic safeguards before querying the LLM. • Downgrades illegal delegation attempts back to the planning phase.

Table 3.10: Node description for lesson specification extraction.

Field	Description
ID	<code>extract_lesson_spec</code>
Agent	Lesson Director
Role	• Extracts the teacher’s lesson objectives and actionable teaching procedures from the chat history.
Execution Trigger	• Runs when the director routing selects the planning branch.
Context & Outcomes	• Reads the trimmed conversation and media signals. • Outputs the persistent <code>lesson_plan</code> and <code>procedure</code> variables into the shared state.

Table 3.11: Node description for input comprehensiveness assessment.

Field	Description
ID	<code>assess_input_comprehensiveness</code>
Agent	Lesson Director
Role	• Decides whether the extracted lesson specification is detailed enough to generate a compliant creation plan.
Execution Trigger	• Runs immediately after lesson specification extraction.

Table 3.11: Node description for input comprehensiveness assessment (Continued)

Field	Description
Context & Outcomes	• Requires at least one objective and one actionable teaching step. • Outputs a readiness rationale and a list of missing requirements if incomplete.

Table 3.12: Node description for plan drafting.

Field	Description
ID	<code>draft_plan</code>
Agent	Lesson Director
Role	• Drafts the canonical <code>creation_plan</code> that outlines the exact scene-by-scene structure of the presentation.
Execution Trigger	• Runs only when the input assessment node confirms the lesson information is comprehensive.
Recovery	• Makes up to three generation attempts, passing validation errors back to the LLM for self-correction. • Surfaces unresolvable structural issues directly to the teacher.

Table 3.13: Node description for developer delegation.

Field	Description
ID	<code>delegate_to_developer</code>
Agent	Lesson Director
Role	• Prepares the state envelope required to trigger the Lesson Developer subgraph.
Execution Trigger	• Runs only when the director possesses both a structurally sound creation plan and confirmed user generation intent.
Context & Outcomes	• Records metadata such as the request ID and start time. • Clears residual payloads and signals the root graph to transition to the developer.

Table 3.14: Node description for lesson creation.

Field	Description
ID	<code>create_lesson</code>
Agent	Lesson Developer
Role	• Materializes the abstract <code>creation_plan</code> into a fully populated, renderer-compatible array of scenes.
Execution Trigger	• Runs as the entry point of the developer subgraph.

Table 3.14: Node description for lesson creation (Continued)

Field	Description
Recovery	• Executes four strict generation attempts with primary and recovery prompts. • Falls back to a relaxed game-prop validation attempt if minor formatting issues persist.

Table 3.15: Node description for the developer report.

Field	Description
ID	<code>report_to_director</code>
Agent	Lesson Developer
Role	• Summarizes the generation outcome into an internal report used by the director to inform the teacher.
Execution Trigger	• Runs immediately after scene creation and acts as the terminal node of the developer subgraph.
Context & Outcomes	• Closes the open generation request and sets the success or failure status. • Routes execution control back to the director.

Table 3.16: Node description for teacher reply composition.

Field	Description
ID	<code>reply_to_user</code>

Table 3.16: Node description for teacher reply composition (Continued)

Field	Description
Agent	Lesson Director
Role	Composes the final message that the teacher reads, summarizing drafted plans, requesting clarifications, or presenting final generation results.
Execution Trigger	Runs after the developer report returns, or when the director decides to reply instead of generating.
Context & Outcomes	- Enforces media-grounding rules and prevents the AI from using incorrect "in-progress" phrasing once generation is complete. - Outputs the canonical assistant response string.

Table 3.17: Node description for turn completion.

Field	Description
ID	finish_turn
Agent	Lesson Director
Role	Safely terminates a director turn using default routing values.
Execution Trigger	Runs when a routing hop limit is exceeded or when duplicate generation is blocked.

Table 3.17: Node description for turn completion (Continued)

Field	Description
Context & Outcomes	• Emits safe defaults (empty messages, idle statuses) without making an LLM call.

Shared State

The graph’s nodes interact via a shared state object, acting as the working memory for the generation process. This state stores structured data rather than raw prompt strings.

Table 3.18: The core shared state values in the multi-agent architecture.

State Value	Description
Conversation history	The sequential chat log between teacher and assistant.
Textbook context	Extracted curriculum passages used to ground the AI’s content.
<code>lesson_plan</code>	The extracted learning objectives derived from the conversation.
<code>procedure</code>	The step-by-step teaching phases envisioned by the teacher.
<code>creation_plan</code>	The canonical outline ensuring structural constraints before generation.
<code>scenes_data</code>	The finalized array of playable slides and activities.
<code>generation_status</code>	The lifecycle state (for example idle, in progress, success) dictating UI behavior.

The `creation_plan` is the most critical state variable. It dictates the sequence of content and interactive scenes, bridging the gap between conversational intent and frontend constraints.

3.2.3 RAG Architecture

Retrieval-Augmented Generation (RAG) augments the underlying language model with external knowledge, ensuring the generated lessons align with official textbook materials [?].

End-to-End Pipeline

The RAG pipeline operates in three distinct phases: ingestion, storage, and retrieval. During ingestion, raw PDFs or markdown files are segmented by headings and processed by a low-temperature language model to produce structured chunk records. These chunks categorize the text by skill type, unit, and lesson index.

Once ingested, the backend generates vector embeddings for each chunk and persists them in the PostgreSQL database using the `pgvector` extension. During a lesson authoring session, the system performs a similarity search against the teacher’s query, extracting the most relevant passages. These passages form the **textbook context** injected into the multi-agent shared state, effectively grounding the Lesson Director and Developer nodes in verified curriculum evidence.

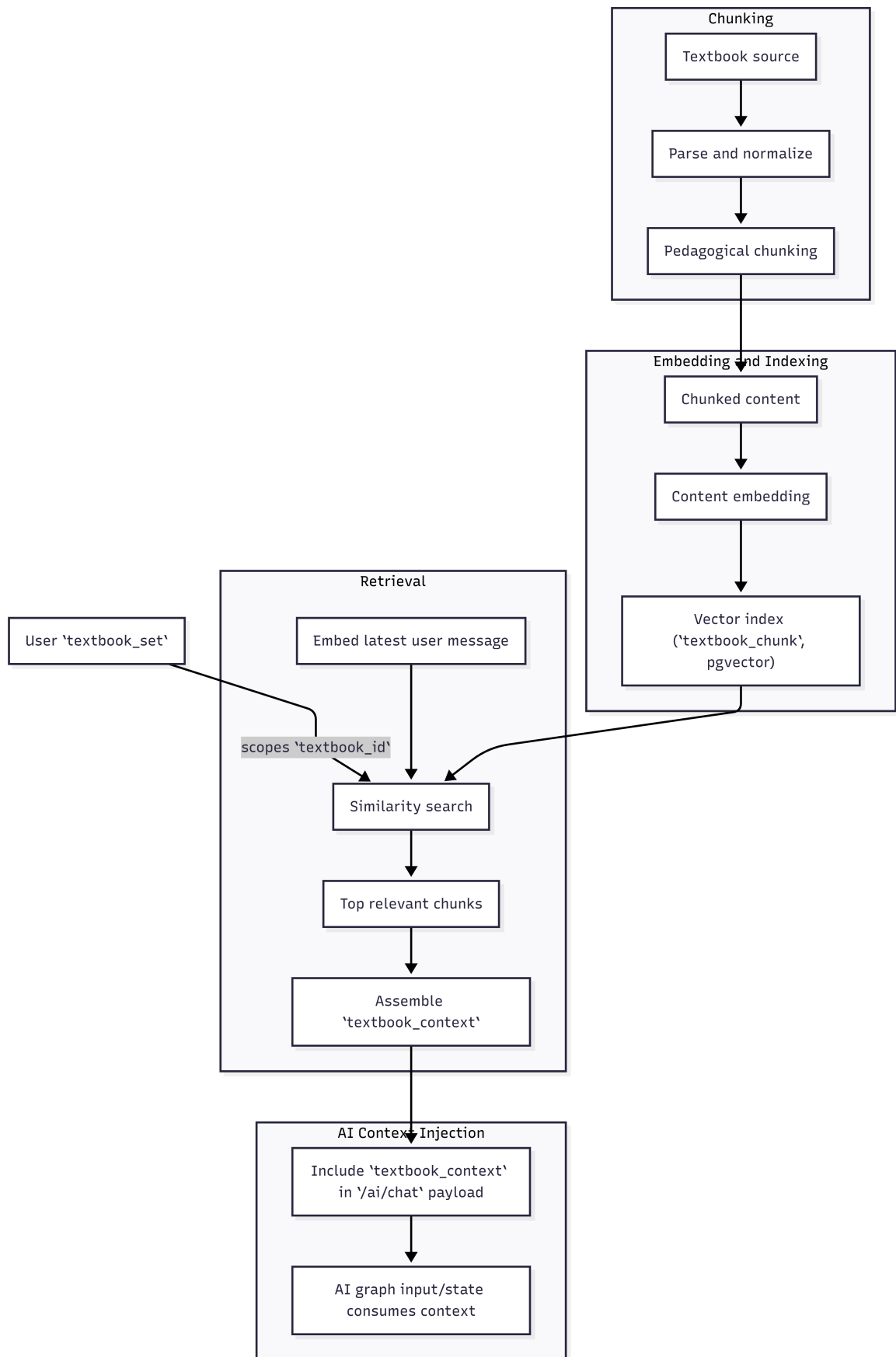


Figure 3.6: Edlora's textbook RAG pipeline.

3.2.4 Backend and Database Architecture

The product backend operates as the bridge between the web client, the PostgreSQL database, and the external AI service.

HTTP Endpoint Catalog

The product API is structured into several core domains. Table 3.19 summarizes the primary HTTP surface.

Table 3.19: Catalog of primary HTTP endpoints.

Method	Path	Auth		Role
GET	/api/auth/me	Bearer quired	re-	Returns the current database user profile, synchronizing from Clerk when necessary.
PATCH	/api/auth/me/onboarding	Bearer quired	re-	Updates the teacher's educational level and textbook set preferences.
GET	/api/auth/check	Bearer tional	op-	Reports authentication success status without throwing an error for invalid tokens.
GET	/api/users/{user_id}	None		Reads a public user profile by identifier.
GET	/api/users/	Bearer quired	re-	Returns a paginated list of active users.
POST	/api/webhooks/clerk	Svix signature		Processes Clerk lifecycle events to automatically create or delete local user rows.

Method	Path	Auth	Role
GET	/api/lessons	Bearer quired	re- Lists the calling teacher's lessons, ordered chronologically.
POST	/api/lessons/create	Bearer quired	re- Creates a new empty lesson. Requires an idempotency key to prevent duplicate creations.
GET	/api/lessons/{lesson_id}	Bearer quired	re- Fetches the full detailed payload of a specific lesson.
PATCH	/api/lessons/{lesson_id}	Bearer quired	re- Applies partial updates to the lesson name, background media, or scene data.
DELETE	/api/lessons/{lesson_id}	Bearer quired	re- Deletes the lesson and cascades the deletion to its scenes.
POST	/api/lessons/chat	Bearer quired	re- Proxies the conversational prompt to the AI service and streams back Server-Sent Events.
POST	/api/storage/upload	Bearer quired	re- Accepts multipart form uploads, returning the public URL and file metadata upon successful storage.

Conceptual Relational Model

The database schema utilizes SQLAlchemy to map application logic to relational tables. The core entities focus on ownership, lesson hierarchy, and curriculum retrieval.

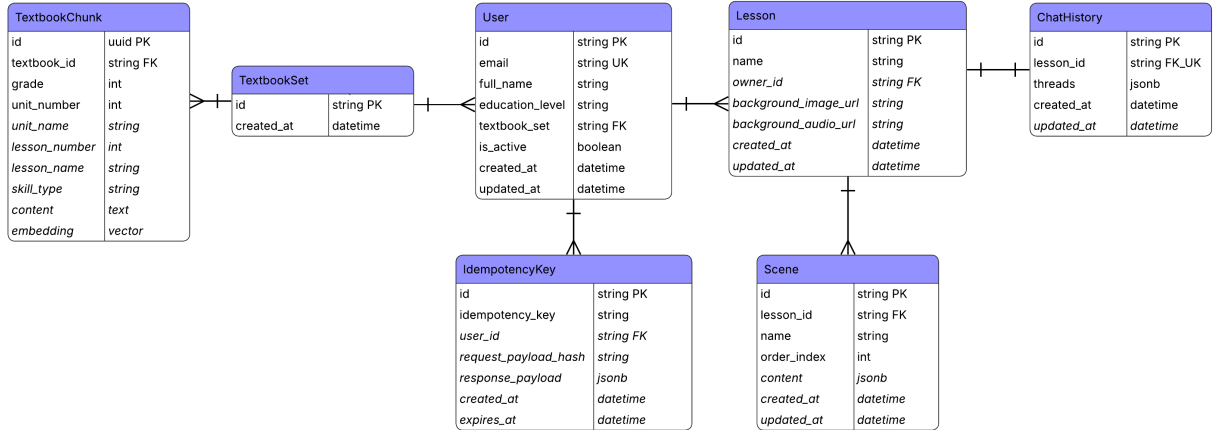


Figure 3.7: Conceptual relational model mapping users, lessons, scenes, and textbook chunks.

The following tables detail the specific columns and constraints for the primary database entities.

Table 3.20: Columns for the synchronized user records.

Column	Type	Constraints	Meaning
id	string	primary key	The Clerk user identifier reused as the local primary key.
email	string	unique, indexed	The login identity synchronized from Clerk.
full_name	string, nullable	nullable	The teacher's display name.
education_level	string, nullable	nullable	The target teaching level captured during onboarding.

Column	Type	Constraints	Meaning
textbook_set	string, nullable		A logical link to the textbook corpus used to filter RAG queries.
is_active	boolean		A soft lockout flag to disable access without deleting data.

The user table is designed to be the central anchor for identity and preferences within the product. Instead of managing complex authentication states, it strictly mirrors the necessary fields from the Clerk provider, ensuring the system always knows the teacher’s baseline configuration before launching into the lesson authoring workflows.

Table 3.21: Columns for top-level lessons edited in the workspace.

Column	Type	Constraints	Meaning
id	string	primary key	A generated UUIDv7 identifier.
name	string		The display title of the lesson.
owner_id	string	FK to user.id	Enforces strict data isolation so teachers only see their own work.
background_image_url	string, nullable		A fallback image URL applied across the entire presentation.
background_audio_url	string, nullable		A fallback audio URL for ambient classroom sound.

Table 3.22: Columns for ordered slides or scenes.

Column	Type	Constraints	Meaning
id	string	primary key	A generated UUIDv7 identifier.
lesson_id	string	FK to lesson.id	Links the scene to its parent lesson.

Column	Type	Constraints	Meaning
<code>name</code>	string		A helpful label for the scene.
<code>order_index</code>	integer		Controls the chronological sort order within the presentation.
<code>content</code>	JSONB		The structured payload defining the slide text or interactive game parameters.

The database architecture deliberately normalizes the presentation into two separate tables: lessons and scenes. The lesson table manages the high-level metadata, such as titles and default background themes. Meanwhile, the scene table stores the actual slide content inside a flexible JSONB column. This design choice is critical because it allows the frontend renderer to freely introduce new interactive game types or dynamic slide formats without forcing the backend to undergo complex, rigid schema migrations every time a new feature is added.

Table 3.23: Columns for conversational transcripts bound to lessons.

Column	Type	Constraints	Meaning
<code>id</code>	string	primary key	A generated UUIDv7 identifier.
<code>lesson_id</code>	string	FK to <code>lesson.id</code>	Binds the transcript to exactly one lesson.
<code>threads</code>	JSONB		An array containing the sequential message log between the teacher and the AI.

The chat history is isolated into its own table rather than being embedded directly inside the main lesson row. Conversational transcripts grow rapidly with every message sent during the authoring process. By separating this heavy data, the system ensures that querying a teacher’s lesson list remains incredibly fast, as the database does not have to load massive, irrelevant chat logs just to display the lesson titles on the dashboard.

Table 3.24: Columns for chunked textbook excerpts and embeddings.

Column	Type	Constraints	Meaning
<code>id</code>	string	primary key	A generated UUID identifier.
<code>textbook_id</code>	string	indexed	Matches the teacher’s onboarding selection to filter the scope of the retrieval.
<code>unit_name</code>	string, nullable	nullable	The human-readable name of the curriculum unit.
<code>skill_type</code>	string, nullable	nullable	Describes the specific competency emphasized in the excerpt.
<code>content</code>	text		The raw, human-readable textbook passage returned to the AI.
<code>embedding</code>	<code>vector(1536)</code>	IVFFlat index	The vector representation used for cosine similarity searching.

The textbook chunk table is fundamentally designed for rapid similarity search. By utilizing the `pgvector` extension, the database can mathematically compare the teacher’s chat query against thousands of curriculum embeddings in mere milliseconds. Additionally, this table is completely decoupled from individual user rows via foreign keys, meaning a single ingested textbook can serve thousands of teachers simultaneously without needlessly duplicating the heavy vector data across the system.

3.3 Implementation

3.3.1 Use Case: Authentication

As established in Section 3.1.1, the **Authenticate (UC-01)** use case ensures the system can verify teachers’ identities to **remember their work and enforce strict data isolation**. This gatekeeping step relies on Clerk [?] for hosted identity management and the product API for backend synchronization, ensuring

that teachers cannot personalize the educational product before their identity is securely established.

Placeholder screenshot: Clerk hosted sign-in (or sign-up) on `/sign-in` or `/sign-up`, centered card, routing `path`, link between flows. Replace with production capture.

Figure 3.8: Hosted Clerk sign-in interface for establishing a teacher session.

Placeholder sequence diagram (vector PDF): Teacher, Browser, Clerk, Next.js, ProductAPI, Postgres. Flow: hosted sign-in, optional `/api/auth/check` or `/api/auth/me`, `/api/auth/me` with JWT and user row, optional webhook POST `/api/webhooks/clerk`. Replace with redrawn diagram from engineering docs.

Figure 3.9: Main success flow for Clerk session establishment and backend user alignment.

The authentication flow begins when the teacher **initiates the sign-in or sign-up process** through the web client, rendering the centered Next.js [?] card interface shown in Figure 3.8. Following this, the browser completes the hosted authentication against Clerk until a **valid session is established and a bearer token is acquired**. As illustrated in Figure 3.9, the Next.js application then uses this token to make protected API calls. The product API **verifies the JSON Web Token (JWT) and subsequently loads or creates the corresponding user row in the PostgreSQL database**. Concurrently, Clerk may trigger asynchronous webhooks (POST `/api/webhooks/clerk`) to update user lifecycle events without blocking the teacher’s interactive experience.

3.3.2 Use Case: Onboarding

Following authentication, the **Onboard (UC-02)** use case requires the teacher to **capture the target education level and official textbook materials**. This ensures that future AI recommendations and chat contexts are **accurately grounded in the appropriate curriculum**, ultimately unlocking the platform’s full lesson capabilities.

Placeholder screenshot: onboarding flow (education level step, textbook set step, personalization loading messages). Headers should read as in production. Replace with production capture.

Figure 3.10: Onboarding wizard for education level and textbook set before accessing the lesson library.

Placeholder sequence diagram (vector PDF): Teacher, Browser, Next.js, ProductAPI, Postgres. PATCH /api/auth/me/onboarding; validate body; update user; return response. Note: async ingestion separate. Replace with redrawn diagram.

Figure 3.11: Synchronous onboarding update for education level and textbook set.

This flow activates when the system’s middleware **detects missing profile fields**, automatically redirecting the teacher to the onboarding wizard depicted in Figure 3.10. The teacher sequentially **selects their specific education level and textbook set**. Upon submission, the Next.js application forwards a PATCH request containing these choices, as mapped out in Figure 3.11. The FastAPI [?] backend then **validates the payload and updates the user record within PostgreSQL**. Finally, the system plays a brief personalization sequence before **redirecting the teacher to the main lesson library**, while the official textbook content is processed in the background for future retrieval-augmented generation.

3.3.3 Use Case: Lesson Management

The **Manage lessons (UC-03)** use case provides the foundational interface for teachers to **view all owned lessons, initialize new drafts, or remove obsolete materials**. This chronological overview ensures that the teacher’s collection strictly matches their intent, preparing the content for deeper editing work.

Placeholder screenshot: `/lessons` lesson library with header *My Lessons*, lesson count, *Create Lesson* button, grid of lesson cards (title, scene count, delete). Empty state variant optional. Replace with production capture.

Figure 3.12: Lesson library listing, creation entry point, and per-lesson navigation to the editor.

Placeholder sequence diagram (vector PDF): Teacher, Browser, Next.js, ProductAPI, Postgres. GET list; alt POST create with idempotency or DELETE owned lesson; GET single lesson before editor hydrate. Replace with redrawn diagram.

Figure 3.13: Lesson list, scaffolded creation, optional delete, and load-for-edit.

The management flow initiates when the teacher **opens the lesson library**, prompting the client to fetch existing lessons from the backend and render them as a responsive grid, as seen in Figure 3.12. To create a new draft, the teacher clicks the primary creation button, causing the client to **send a POST request equipped with an idempotency key to prevent duplicate rows**, as detailed in Figure 3.13. The API **persists a scaffolded lesson row and immediately navigates the teacher to the authoring editor**. Alternatively, the teacher can **select an existing lesson for deletion**, which sends a DELETE request to safely remove the owned data and refreshes the client state to accurately reflect the updated backend.

3.3.4 Use Case: Lesson Creation

Building upon lesson management, the **Edit lesson (UC-04)** use case allows the teacher to **transform a partial lesson draft into a story-driven experience**. This process involves configuring goals, arranging scenes, uploading media, and interacting with the teaching assistant to incrementally assemble the presentation.

Placeholder screenshot: lesson editor at `/lessons/{lessonId}`, three-column layout (scenes, preview with *Play*, chat with messages and input). Replace with production capture.

Figure 3.14: Authoring canvas with scenes, preview, and assistant chat.

Placeholder sequence diagram (vector PDF): Teacher, Browser, Next.js, ProductAPI, Postgres, Cloudflare R2, AIModule. GET lesson; optional upload; POST chat SSE with optional textbook retrieval loop to AI stream; PATCH lesson for direct edits. Replace with redrawn diagram.

Figure 3.15: Authoring flow with optional upload, streaming chat with optional textbook grounding, and direct PATCH.

The authoring flow starts when the client **fetches the full lesson payload to hydrate the editor's three-column layout**, as demonstrated in Figure 3.14. When the teacher interacts with the teaching assistant, the client **sends a chat request utilizing streaming semantics** [?], heavily detailed in Figure 3.15. The backend then **retrieves relevant textbook chunks from PostgreSQL and forwards the contextual data to the AI module** [?]. The AI module **progressively streams replies via Server-Sent Events (SSE)**, simultaneously updating the visual chat history and the structural lesson scenes. Concurrently, the teacher can **upload multimedia assets to Cloudflare R2 storage or directly patch lesson attributes**, with the system persistently saving all modifications to guarantee continuity across sessions.

3.3.5 Use Case: Lesson Presentation

The final step in the instructional workflow is the **Present lesson (UC-05)** use case, which executes the finished content as a **fullscreen slideshow or interactive story for learners**. This dedicated mode strictly restricts editing capabilities, ensuring a stable and uninterrupted classroom-facing delivery.

Placeholder screenshot: presentation (fullscreen) lesson view showing scene content without editor chrome. Replace with production capture.

Figure 3.16: Presentation view for advancing scenes during teaching.

Placeholder sequence diagram (vector PDF): optional GET lesson; teacher starts play; fullscreen request with denial branch; loop advance scenes locally; exit fullscreen and restore editor. Replace with redrawn diagram.

Figure 3.17: Client-centered presentation with optional lesson refetch before play.

The presentation flow is triggered when the teacher **initiates the Play command directly from the authoring canvas**. If supported by the environment, the browser **requests fullscreen access and transitions the interface into the immersive presentation mode** displayed in Figure 3.16. As outlined in Figure 3.17, the system **relies entirely on local client state to advance through the narrative scenes**, rendering templates natively without requiring additional structural API calls. Upon completion or intentional interruption, the teacher **exits fullscreen mode, gracefully terminating the play session and returning to the standard editor workspace**.

Chapter 4

Results and Evaluation

Following the comprehensive architectural design and technical implementation of the Edlora Lesson Development Environment (LDE) detailed in Chapter ??, this chapter presents a rigorous empirical evaluation of the developed system. The realization of a complex, artificial intelligence-driven application necessitates a multi-dimensional validation approach to ensure that the theoretical objectives translate into functional, reliable, and pedagogically sound software.

The evaluation is systematically structured across three distinct domains. First, the software engineering integrity of the system is measured through exhaustive automated testing procedures, quantifying the stability of the underlying codebase. Second, the generative capabilities of the artificial intelligence pipelines are scrutinized, employing strict mathematical metrics and LLM-as-Judge frameworks to evaluate retrieval accuracy and pedagogical alignment. Finally, the practical utility and user-centric efficacy of the platform are assessed through qualitative end-to-end evaluations conducted with active secondary school educators. Together, these results provide a holistic validation of Edlora’s feasibility and establish a grounded foundation for future iterations.

4.1 Software Testing Results

The fundamental challenge of software quality is ensuring that the implemented system reliably executes its intended behavioral boundaries. To verify that the platform meets the strict operational requirements defined in Section 3.1, the thesis employs a comprehensive, multi-layered software testing strategy. The following sections detail the procedures used to validate the codebase, as well as the objective metrics and results gathered from the execution of these tests.

4.1.1 Procedures and Metrics

The overarching approach adopted for validating the system is the testing pyramid. As illustrated in Figure ??, this structural paradigm organizes software verification into three distinct layers, prioritizing execution speed and isolation to ensure high confidence in the software’s stability.

At the foundation, a large volume of unit tests verifies individual functions and components in isolation. Above this base, a middle layer of integration tests validates the boundaries and interactions between major subsystems. Finally, a narrow top layer of end-to-end (E2E) tests exercises the complete user journey in a simulated production environment.

The choice of testing technologies reflects this structural approach, but the tools themselves are secondary to the software engineering principles they enforce. For the foundational unit and integration layers, the frontend utilizes simulation libraries to replicate a browser environment, while the product backend and AI modules rely on dedicated Python test runners. The primary goal at these lower levels is to ensure robust edge-case and unhappy-path handling. For instance, the tests systematically simulate network rejections, malformed server responses, and unauthorized access attempts. By deliberately targeting these failure states, the system proves it can fail gracefully rather than corrupting user data. To validate complex stateful interactions, the integration tests deliberately cross system boundaries, such as querying a live vector database to verify retrieval accuracy under realistic conditions.

To objectively evaluate the quality of this foundational testing, the methodology tracks two primary metrics: code coverage and branch coverage. Code coverage measures the absolute percentage of source lines executed during a test suite. Branch coverage provides a much stricter assessment by measuring the percentage of executed control-flow branches, ensuring that both the primary logic and the fallback logic are verified.

At the top of the pyramid, the E2E layer utilizes browser automation tools to simulate real user interactions. The core software engineering principle applied here is scenario-first design. Instead of writing tests based on technical implementation details, the tests are authored based directly on the user scenarios established during

the requirements analysis. This ensures that the testing suite explicitly validates the platform’s primary Use Cases. By mapping technical assertions directly to the intended business logic, this approach theoretically proves the system’s production readiness.

4.1.2 Results

The execution of these testing procedures yields objective data regarding the platform’s structural reliability. At the foundational layer, the unit and integration suites achieve substantial coverage across the platform’s core subsystems.

Table 4.1: Coverage results ledger for unit and integration suites.

Subsystem Scope	Measurement Tool- ing	Line Rate	Branch Rate
Product Backend	pytest-cov	86.56%	69.04%
AI Module	pytest-cov	89.12%	73.11%
Web Frontend	Vitest V8	75.91%	63.71%

As detailed in Table 4.1, the coverage metrics purposefully do not reach absolute saturation. Achieving total branch coverage is often impractical in complex, distributed systems. The uncovered branches in the platform primarily belong to environment startup paths, external third-party service boundaries, and complex visual runtime contexts. Reaching total branch saturation on these specific paths would require immense effort dedicated to synthetic fault injection, which yields diminishing returns. Therefore, absolute mathematical perfection is not the goal. Instead, the strategy provides a layered defense where risk-critical boundaries are heavily verified, establishing a high degree of confidence in the logic that directly satisfies the system requirements.

Moving to the top of the testing pyramid, the E2E results evaluate the realistic user experience. Table 4.2 details the requirement validation matrix, tracing the primary Use Cases to specific behavioral scenarios and their latest execution outcomes.

Table 4.2: End-to-end requirement validation matrix.

Use Case	Scenario	Result
UC-02	The system redirects a new user to the education level selection screen.	Failed (Timeout)
	The system persists textbook selections and navigates to the lesson list.	Blocked
UC-03	The user successfully creates a new untitled lesson in the managed list.	Blocked
	The system displays a conflict error gracefully and allows the user to retry.	Blocked
UC-04	The system collects missing input and blocks generation until requirements are met.	Blocked
	The authoring flow reports AI model failures gracefully without silent data corruption.	Blocked
UC-05	The presentation journey allows navigation through generated interactive scenes.	Coverage Gap

A rigorous testing evaluation requires absolute academic honesty regarding system stability. While the E2E design successfully maps to the intended Use Cases, the execution results reveal challenges with infrastructure consistency. In the latest automated evaluation, the E2E suite encountered an immediate timeout failure at the first onboarding navigation step. Because the onboarding process (Use Case 02) strictly gates access to the core application, this single infrastructure timeout blocked the runner from verifying the subsequent scenarios.

This outcome highlights a critical reality in integrated software testing: a failure at an early integration boundary completely blocks the validation of downstream workflows. Consequently, while the scenario structures definitively validate the key requirements, the system currently lacks the reliable pass consistency needed to guarantee flawless end-to-end stability. Resolving these environmental timeouts remains an active engineering objective to fully realize the automated verification of the platform’s production readiness.

4.2 AI Evaluation Results

This section presents the evaluation of the artificial intelligence capabilities within the Edlora platform. The assessment is divided into two primary areas: the generative behaviors of the multi-agent routing system and the retrieval performance of the textbook context pipeline. First, the procedures, tools, and metrics used to measure these systems are defined. Following this, the qualitative and quantitative results are analyzed to determine how well the system satisfies the established behavioral requirements.

4.2.1 Procedures and Metrics

Evaluating generative software requires a different approach than traditional deterministic testing. Because language models produce highly variable outputs, standard unit tests cannot easily determine whether a response is helpful, engaging, or structurally sound. To address this, the evaluation relies on automated judging mechanisms and explicit benchmark datasets.

AI Procedures and Metrics

The generative behavior of the system is evaluated using the LLM-as-a-Judge technique. This approach replaces human raters with a large language model configured to score outputs based on strict grading rubrics. In this project, evaluator functions are created using the OpenEvals library and executed through LangSmith against seeded datasets. To prevent the judge model from becoming confused by excessively large or unstructured data, the input state is serialized. This serialization ensures that the judge only reads bounded and safe fields, such as the `response_message`, the `scenes_data`, and specific excerpts of the durable lesson state.

The evaluation framework is designed around the platform’s multi-agent architecture. Operationally, the evaluation tracks are divided to test the system root, the director routing logic, and the developer routing logic. Table 4.3 highlights the specific technologies and configurations used to execute these evaluation procedures.

Table 4.3: Technologies and tools used for AI evaluation.

Component	Role in Evaluation
Judge factory	Utilizes <code>openevals</code> to define continuous judge scorers from rubric prompts and feedback keys.
Experiment runner	Utilizes <code>langsmith</code> to execute evaluator suites against datasets and store experiment results.
Judge prompts	Encodes rubric criteria for helpfulness, reference alignment, story framing, scene adherence, and storytelling quality.
Dataset orchestration	Ensures reproducible dataset names and seed examples before evaluation to prevent mid-comparison mutation.
Judge input normalization	Limits payload size and enforces stable JSON structures to reduce judge drift from oversized state variables.
Legacy baseline evaluators	Provides non-agentic baseline scoring dimensions for comparative framing through deterministic quality and structure checks alongside an LLM correctness judge.

To understand the value of the routing logic, the system is measured using a baseline comparison protocol. This protocol pits a vanilla model utilizing a legacy non-agentic router against the proposed agentic routing model. Both tracks run on fixed LangSmith datasets to ensure the underlying prompts and expected outcomes remain constant. The judge model identifier is also pinned so that the scoring remains consistent. Finally, the results are aggregated by calculating the mean scores for each rubric axis, which are then mapped to normalized values for direct comparison.

RAG Procedures and Metrics

The retrieval-augmented generation (RAG) pipeline is benchmarked using an explicit query dataset rather than automated generation. The dataset contains five

fixed cases targeting the official Global Success textbook materials. Each case specifies a query, a textbook scope, relevance marker phrases, and interpretation notes. This creates a marker-based benchmark where retrieval success is evaluated by the presence of these specific phrases in the returned text chunks.

The RAG benchmark is executed using a dedicated evaluation script that queries the PostgreSQL database via pgvector. Table 4.4 outlines the technologies that drive this measurement.

Table 4.4: Technologies and tools used for RAG evaluation.

Component	Role in Evaluation
Retrieval engine	Uses pgvector cosine distance queries to produce ranked chunk candidates for each benchmark query.
Embedding model	Utilizes <code>text-embedding-3-small</code> to encode user query text for semantic retrieval.
Metric scorer	Computes Context Precision, Context Recall, Hit Rate, MRR, and latency aggregates based on marker overlap.
Benchmark runner	Executes the benchmark dataset and writes the calculated results to a dedicated JSON artifact file.
Query dataset	Defines the repeatable query set and the specific relevance markers required to validate textbook context.

The performance of the retrieval system is quantified using specific relevance and ranking metrics. In these technical definitions, calculation methods natively incorporate the evaluation conditions. Let R_r represent the count of retrieved chunks containing at least one relevance marker, and let R_t represent the total count of retrieved chunks. The Context Precision is defined as follows:

$$\text{Context Precision} = \frac{R_r}{R_t}$$

Let M_m represent the number of relevance markers that were successfully matched in the retrieved text, and let M_t represent the total number of relevance markers defined for the query. The Context Recall is defined as follows:

$$\text{Context Recall} = \frac{M_m}{M_t}$$

Ranking quality is measured through Hit Rate and Mean Reciprocal Rank (MRR). The Hit Rate calculation method returns 1 if at least one retrieved chunk contains a relevance marker, and 0 otherwise. The MRR calculation method evaluates the reciprocal rank of the first relevant chunk, returning $\frac{1}{\text{rank}}$ if a hit occurs, and 0 otherwise. Finally, performance is measured by observing latency, which calculates the query runtime in milliseconds around the retrieval call and aggregates the results via percentile interpolation.

4.2.2 Results

The results of the evaluation process demonstrate the platform’s ability to balance storytelling engagement with strict pedagogical boundaries. This subsection details the qualitative examples and the quantitative metrics gathered from the experiment runners.

AI Results

The AI module is evaluated across several distinct rubric axes to capture both the conversational quality and the structural fidelity of the generated content. Table 4.5 presents the performance delta between the vanilla baseline model and the proposed agentic routing model across these key evaluation rubrics.

Table 4.5: Baseline comparison of Vanilla versus Agentic models across evaluation rubrics.

Metric Axis	Vanilla Model	Agentic Model	Delta
Reply helpfulness	TBD-LANGSMITH-VANILLA-HELPFULNESS	TBD-LANGSMITH-AGENTIC-HELPFULNESS	TBD
Reference alignment	TBD-LANGSMITH-VANILLA-REFALIGN	TBD-LANGSMITH-AGENTIC-REFALIGN	TBD
Story framing	TBD-LANGSMITH-VANILLA-STORY	TBD-LANGSMITH-AGENTIC-STORY	TBD
Scene adherence	N/A (legacy vanilla does not generate full scenes)	TBD-LANGSMITH-AGENTIC-SCENEPLAN	N/A
Storytelling arc	TBD-LANGSMITH-VANILLA-ARC	TBD-LANGSMITH-AGENTIC-ARC	TBD
Storytelling engagement	TBD-LANGSMITH-VANILLA-ENGAGEMENT	TBD-LANGSMITH-AGENTIC-ENGAGEMENT	TBD

To contextualize these results, each metric axis evaluates a specific behavioral boundary. **Reply helpfulness** checks if the assistant provides clear, actionable guidance rather than generic filler. **Reference alignment** ensures the generated content is strictly grounded in the provided educational materials, preventing hallucinations. **Story framing** evaluates how well the assistant introduces narrative elements when requested, without penalizing necessary expository text. **Scene adherence** verifies that the generated interactive scenes faithfully follow the sequential order and intent of the approved creation plan. Finally, the **storytelling arc**

assesses the overall narrative coherence across multiple scenes, while **storytelling engagement** measures the vividness and age-appropriateness of the writing.

Ultimately, these quantitative improvements demonstrate that the architectural engineering has paid off. By moving from a vanilla router to a specialized multi-agent system, the platform successfully delivers an AI model that is highly optimized for creating structured, engaging, and meaningful educational lessons.

RAG Results

The RAG evaluation utilizes a five-query dataset explicitly scoped to the official textbook materials. The dataset features relevance marker annotations designed to test both general retrieval and unit-filtered retrieval accuracy.

Table 4.6 presents the resulting metrics from the retrieval benchmark. Currently, the execution of this benchmark requires a valid external provider key, and the documented run reflects an intentionally skipped status pending the final production environment configuration.

Table 4.6: Retrieval evaluation metrics from the benchmark dataset.

Metric	Value
Context Precision	Not measured in this run (benchmark skipped)
Context Recall	Not measured in this run (benchmark skipped)
Hit Rate	Not measured in this run (benchmark skipped)
MRR	Not measured in this run (benchmark skipped)
Latency (Mean)	Not measured in this run (benchmark skipped)
Latency (p50)	Not measured in this run (benchmark skipped)
Latency (p95)	Not measured in this run (benchmark skipped)
Latency (p99)	Not measured in this run (benchmark skipped)

These outcomes demonstrate that the RAG framework is fully implemented and capable of measuring retrieval quality and latency in a reproducible manner. While the quantitative retrieval performance claims are pending the final automated re-run, the implemented methodology satisfies the requirement that the system can retrieve relevant textbook context to support truthfulness and ground the gener-

ated teaching presentations.

Chapter 5

Conclusion and Future Work

The culmination of this research marks a critical juncture in the intersection of software engineering, artificial intelligence, and cognitive pedagogy. This chapter synthesizes the fundamental findings of the thesis, evaluates the success of the Edlora Lesson Development Environment (LDE) against its initial objectives, and delineates a strategic roadmap for the platform’s continuous evolution. Like any sophisticated piece of software, Edlora is not a static artifact but a dynamic ecosystem. It must continuously adapt to emerging technological capabilities and pedagogical insights until it fully realizes its vision of empowering every educator to effortlessly create inspiring knowledge journeys.

5.1 Conclusion

The genesis of this thesis was rooted in a profound educational tragedy: the systematic burnout of educators caused by the immense operational friction of lesson planning. As the global economy pivots toward requiring critical thinking and conceptual mastery, the educational paradigm must shift from rote memorization to deep, narrative-driven understanding. Cognitive science definitively proves that embedding knowledge within storytelling arcs and active gamification loops maximizes retention and student engagement. However, realizing this pedagogical ideal requires an unsustainable investment of time from educators, who are currently hindered by a fragmented ecosystem of siloed, pedagogically unaware software tools like Microsoft PowerPoint, generic ChatGPT, and Canva.

To resolve this systemic bottleneck, this thesis introduced the conceptual and architectural framework for Edlora, the world’s first Lesson Development Environment. Drawing parallels to Integrated Development Environments (IDEs) in software engineering, Edlora was designed to provide a unified, context-aware suite of tools engineered specifically for educators.

The primary objective was to develop a robust Proof of Concept (POC) focus-

ing on the most resource-intensive artifact: the electronic teaching presentation. This was achieved through a meticulously engineered, multi-tier distributed architecture. By implementing a Retrieval-Augmented Generation (RAG) pipeline utilizing `pgvector` and OpenAI embeddings, the system successfully eliminated AI hallucination, ensuring all generated content was strictly anchored to the mandated "Global Success" secondary school curriculum.

Furthermore, the integration of an Agentic Artificial Intelligence workflow via LangGraph transformed the system from a passive text generator into an active co-teacher. The AI agents were strictly prompted with cognitive science principles, allowing them to autonomously synthesize curriculum facts into cohesive narratives and interactive gamified assessments. Crucially, the architectural decision to decouple logical generation from spatial formatting via a proprietary JSON Flat Block Map and a dynamic Next.js React rendering engine eliminated the tedious necessity for pixel-perfect manual formatting.

The empirical evaluations detailed in Chapter 4 conclusively demonstrated the feasibility and efficacy of this architecture. Exhaustive Test-Driven Development (TDD) protocols, including unit testing and Playwright End-to-End (E2E) verification, confirmed the structural stability of the asynchronous streaming pipelines. LLM-as-Judge evaluations verified that the generated outputs adhered strictly to formatting schemas while successfully integrating narrative and gamification elements. Most importantly, the qualitative user evaluations conducted with active educators validated the platform's product-market fit. The educators confirmed that the system drastically reduced their operational burden while elevating the pedagogical quality of their presentations, effectively validating the core hypothesis of this research.

5.2 Limitations of the Current Research

Despite the successful demonstration of the core concepts, the current iteration of the Edlora POC possesses several constraints that must be acknowledged. A rigorous academic evaluation requires transparency regarding the boundaries of the developed system, which subsequently informs the trajectory of future engineering efforts.

The primary limitation resides in the scope of the implemented artifact. To maintain a feasible engineering lifecycle for a graduation thesis, the implementation was strictly constrained to the generation of the teaching presentation. A complete lesson, however, is a triad consisting of a lesson plan, the presentation, and the student assessment. While the underlying database schema and vector infrastructure were designed to support this full context, the current user interface and agentic workflows do not yet facilitate the seamless, end-to-end generation of all three synchronized artifacts.

Secondly, the system currently exhibits observable latency during complex generation cycles. While the implementation of Server-Sent Events (SSE) successfully mitigates perceived wait times for conversational chat, the final orchestration of the Presentation Director Agent—which requires synthesizing RAG context, formatting complex JSON block maps, and executing multiple LLM reasoning steps—introduces a psychological friction point. As noted during the educator evaluations, this latency momentarily disrupts the user’s flow state.

Finally, the system is presently hard-coupled to the OpenAI ecosystem (specifically GPT-4o and its variants). While these models offer state-of-the-art reasoning capabilities, relying on a single proprietary vendor introduces systemic fragility and economic rigidity. The high inference costs associated with flagship models currently limit the economic scalability of the platform for broad, democratized access in lower-income educational districts.

5.3 Future Work

The conclusion of this thesis does not represent the termination of the Edlora project, but rather the completion of its foundational architecture. To fully realize the strategic vision of "One Lesson, One Knowledge Journey," a comprehensive development roadmap has been established, categorized into immediate short-term optimizations and expansive long-term innovations.

5.3.1 Short-Term Roadmap

The immediate priority for the next development cycle is addressing the feedback derived from the qualitative educator evaluations and completing the core LDE

triad.

To resolve the psychological friction caused by systemic latency, the frontend rendering engine will be upgraded to implement predictive skeleton loaders and optimistic UI updates within the Live Preview panel. By visually communicating the exact structural blocks the AI is actively assembling before the final text payload resolves, the perceived latency will be drastically reduced. Additionally, backend optimization of the LangGraph state transitions will be conducted to parallelize independent agentic tasks, reducing the overall time-to-first-byte for complex presentation payloads.

From a functional perspective, the system will be expanded to natively generate and manage Lesson Plans and Assessments. Because the foundational PostgreSQL database and `pgvector` RAG pipeline already possess the complete curriculum context, new agentic workflows will be engineered to synthesize these artifacts simultaneously. When an educator requests a presentation change via the chat interface, an overarching "Consistency Agent" will automatically propagate those modifications across the associated lesson plan and assessment documents, truly realizing the promise of seamless change management.

To address economic scalability and vendor lock-in, the backend LLM orchestration layer will be refactored utilizing LangChain's model-agnostic interfaces to support multiple foundation models. By integrating alternative models such as Anthropic's Claude 3.5 Sonnet or Google's Gemini Pro, the system will employ dynamic model routing. Simple formatting tasks will be routed to faster, cheaper open-source models (e.g., Llama 3 running on dedicated inference endpoints), reserving the expensive flagship models strictly for complex pedagogical reasoning.

5.3.2 Long-Term Vision

The long-term trajectory of Edlora involves evolving from a localized presentation generator into a universally adaptable, multimodal educational ecosystem.

Currently, the storytelling framework relies heavily on text and pre-existing static imagery. Future iterations will integrate multimodal generative models, allowing the AI to autonomously generate highly specific, context-aware imagery and audio narrations that align perfectly with the generated story arcs. For instance, if an

educator prompts a lesson about the solar system, the system will not only generate the pedagogical text but also instantly render anatomically correct, stylized visual assets to accompany the narrative, further reducing the educator's operational burden.

Ultimately, the grand vision for Edlora is to achieve true student-level personalization. Currently, the RAG pipeline is grounded in the macro-level context of a textbook. In the future, the vector database will be expanded to ingest micro-level student data, including individual learning styles, phonetic weaknesses, and historical engagement metrics. The AI co-teacher will dynamically adjust the vocabulary difficulty, gamification mechanics, and narrative themes of a lesson not just for the entire class, but personalized for individual student tablets in real-time. By continuously bridging the gap between cutting-edge technological infrastructure and deeply human cognitive pedagogies, Edlora will continue to empower educators to cultivate profound, meaningful learning experiences for generations to come.

References

- [1] Gordon H. Bower and Michal C. Clark. Narrative stories as mediators for serial learning. *Psychonomic Science*, 14(4):181–182, 1969.
- [2] Thomas Graeber, Christopher Roth, and Florian Zimmermann. Stories, statistics, and memory. *Quarterly Journal of Economics*, 2024.
- [3] Juho Hamari, Jonna Koivisto, and Harri Sarsa. Does gamification work? – A literature review of empirical studies on gamification. *47th Hawaii International Conference on System Sciences*, pages 3025–3034, 2014.
- [4] Muhammad Jihadillah. The effectiveness of gamification-based learning in science education. *Journal of Educational Sciences*, 2025.
- [5] Enkelejda Kasneci et al. Chatgpt for good? on opportunities and challenges of large language models for education. *Learning and individual differences*, 103:102274, 2023.
- [6] Chris Kyriacou. Teacher stress: Directions for future research. *Educational review*, 53(1):27–35, 2001.
- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [8] Meaghan Lister. Gamification: The effect on student motivation and performance at the post-secondary level. *Issues and Trends in Educational Technology*, 3(2), 2015.
- [9] Richard E. Mayer. *The Cambridge Handbook of Multimedia Learning*. Cambridge University Press, 2nd edition, 2014.
- [10] Roger C. Schank and Robert P. Abelson. Knowledge and memory: The real story. *Knowledge and Memory: The Real Story*, pages 1–85, 1995.
- [11] World Economic Forum. Future of jobs report 2023. Technical report, World Economic Forum, 2023.